

In the name of God

University of Tehran

School of Electrical and Computer Engineering



Introduction to Machine Learning

Final Project Report

Detection of Speaker Gender and Identity

Supervised by:
Dr. A. Dehaqani, Dr. N. Araabi

group members	student IDs
Kasra Hajiheidari	810199400
M. Mahdi Doroudiyan	810199417
Arshia Abolghasemi	810199319
Shahab Rakhsha	810199423

Fall 2025

Contents

Abstract 3

Preprocessing 4

- Correcting Format 4
- Loading data 6
- Resampling voices 8
- Denoising voices 9
- Frame Segmentation (Windowing) 10
- feature extraction 12
  - 1. Log Mel Spectrogram 12
  - 2. MFCC (Mel Frequency Cepstral Coefficients) 12
  - 3. Spectral Centroid 13
  - 4. Spectral Bandwidth 13
  - 5. Spectral Contrast 13
  - 6. Zero-Crossing Rate 13
  - 7. Energy 14
- Visualization 17

Classification 21

- Gender Classification 21
  - Normalization 26
  - Training Models 26
  - The result of each model 30
- Identity Recognition (Closed-Set Authentication) 32
  - Normalization 34
- Training Models 34
  - The result of each model 35

Clustering 41

- K-Means Clustering : 43
- Agglomerative Clustering 50
- DBSCAN 55

Abstract

Machine learning is essential for analyzing data patterns, particularly in **speaker identification** and **authentication**. This project focuses on two key areas: speaker gender classification and identity verification through audio features. The model predicts a speaker's gender using acoustic attributes and authenticates their identity based on voice characteristics, leveraging a diverse dataset that captures variations in speech patterns. Key steps include **feature extraction**, **noise reduction**, **handling irrelevant or misleading attributes**, and the evaluation of different machine learning algorithms to improve accuracy. By tackling real-world challenges like data variability and noise, we have gained practical experience that enhances our understanding of core machine learning concepts and provides hands-on practice in speech processing and classification.

Preprocessing

Correcting Format

As we know, the standard format for naming the voices is as follows:

HWx_Qy_Student-ID_gender.mp3

First, we had to extract the format of each voice according to the way the voices were named. For this purpose, we start separating according to the separators inside each data's name. From the extracted formats, we create a data set called metadata, the first column of which is the file name, the second column is the question number, the third column is the student ID, the fourth column is the gender, the fifth column is the path, and the last one is has_intro.

```
def check_and_extract_metadata(directory):
    metadata_list = []
    incorrect_files = []
    for filename in os.listdir(directory):
        if filename.endswith(".mp3"):
            parts_underline =
filename.split(".")[0].split("_")
            parts_dash =
filename.split(".")[0].split("-")
            parts_dot =
filename.split(".")[0].split(".")
            if(len(parts_underline) == 4):
                parts = parts_underline
            elif(len(parts_dash) == 4):
                parts = parts_dash
            elif(len(parts_dot) == 4):
                parts = parts_dot
            else:
                incorrect_files.append(filename)
                continue
            student_id = parts[2]
            question = parts[1]
            gender = parts[3]
            path = os.path.join(directory, filename)
            metadata_list.append({
                "filename": filename,
                "question_number": question,
                "student_id": student_id,
                "gender": gender,
                "path": path
            })
    return pd.DataFrame(metadata_list) , incorrect_files
```

According to the data extracted from the data set, some of the voices don't have the standard format and are not named correctly. So correct these formats by using different methods like `.apply` syntax. For data that the function can extract completely, in the gender column, we have the following information:

```
metadata_df['gender'].value_counts()
```

```
gender
male                              541
female                            181
Male                               15
MALE                                8
810101423_male                      1
gender                              1
male-converted                      1
male (online-audio-converter        1
male                                1
male 2                              1
```

So we can do these to replace ‘Male’, ‘MALE’, or another fault with a male:

```
metadata_df['gender'] = metadata_df['gender'].apply(lambda x: 'male' if x == 'Male' or x == 'MALE' else x)
metadata_df['gender'] = metadata_df['gender'].apply(lambda x: 'male' if x == 'gender' else x)
metadata_df['gender'] = metadata_df['gender'].apply(lambda x: 'female' if x == 'female' else 'male')
```

after these replacements, the gender column is cleared.

```
metadata_df['gender'].value_counts()
```

```
gender
male      570
female    181
```

In the student_ID column, we have the following information:

```
metadata_df['student_id'].value_counts()[metadata_df['student_id'].value_counts() < 6].index.tolist()
```

```
['810103092','810103344','810103338','810100217','810199459','810102327','810103226',
'810103136','810103245','810103339','81010','810199319','610399206','Student']
```

Replace the record where the student ID is imperfect:

```
metadata_df[metadata_df['student_id'] == 'Student']
```

filename	question_number	student_id	gender	path
HW1-intro-Student-810101423_male.mp3	intro	Student	male	..\dataset\HW1_M\HW1-intro-Student-810101423_m...

```
metadata_df['student_id'] = metadata_df['student_id'].apply(lambda x: '810101423' if x == 'Student' else x)
```

But we save the second incomplete data for gender detection. For other data the value count of them is less than 6 will be checked in `metadata_incorect_df`.

After these changes, `metadata_df` is as follows:

```
metadata_df.info()
```

```
RangeIndex: 751 entries, 0 to 750

Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   filename         751 non-null    object
 1   question_number  751 non-null    object
 2   student_id       751 non-null    object
 3   gender           751 non-null    object
 4   path             751 non-null    object
```

we do this process for `metadata_incorect_df` to clear all data. In some cases set the format manually.

After all of the process concatenates these two datasets, the result is as follows:

```
metadata_df.info()
```

```
RangeIndex: 822 entries, 0 to 821
Data columns (total 6 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   filename         822 non-null    object
 1   question_number  822 non-null    object
 2   student_id       822 non-null    object
 3   gender           822 non-null    object
 4   path             822 non-null    object
 5   has_intro        822 non-null    int64
```

At the end write it in a .csv file:

```
metadata_df.to_csv('metadata.csv', index=False)
```

We now have a clear dataset of our voice metadata that we can use for training the network.

Loading data

By using Librosa library load the raw audio file. For example, for audio of a male group, the waveform, spectrogram (STFT according to time in dB/sec scale), mel spectrogram, and Fourier transform are as follows:

```
audio_path = metadata['path'][0]
y, sr = librosa.load(audio_path, sr=None)
```

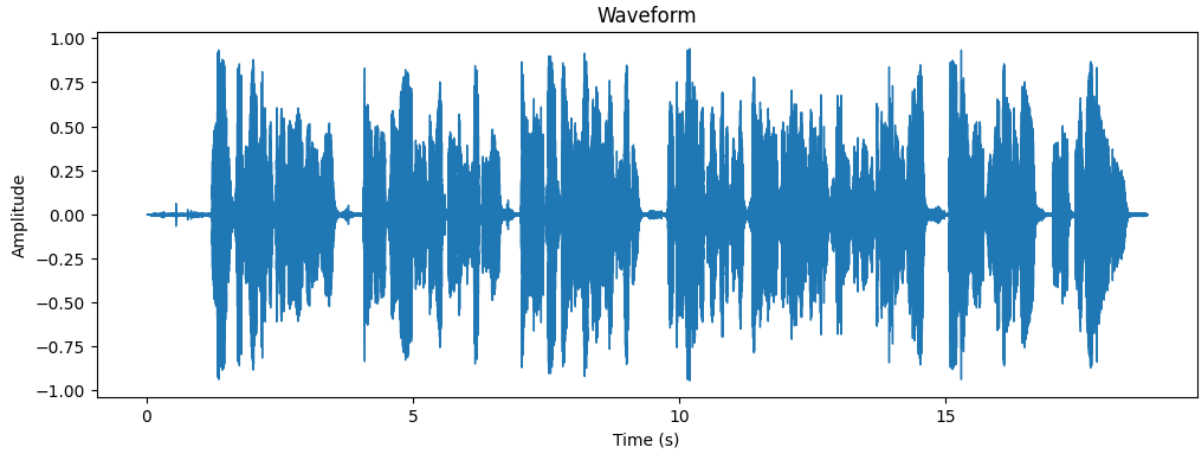


Figure 1: Waveform of a male audio sample

The waveform represents the amplitude of a sound wave over time. The amplitude ranges from -1.00 to 1.00 (cause of normalization), indicating the loudness or intensity of the sound at different points in time. The variations in amplitude suggest changes in the volume and intensity of the male voice. Based on fluctuations in silent parts, there is background noise, so the voice needs to be cleared of that noise.

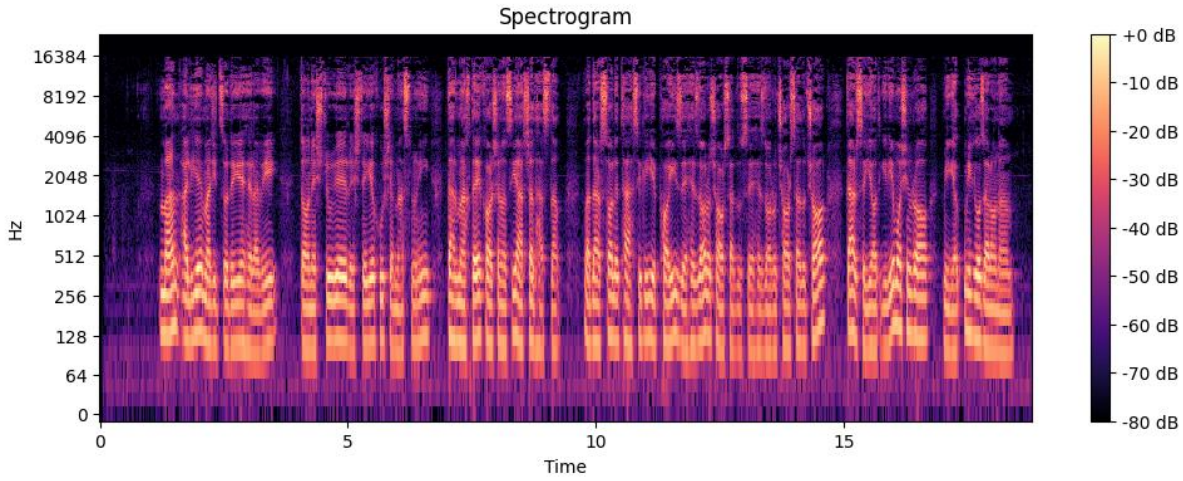


Figure 2: Spectrogram of a male audio sample

Male voices generally have a fundamental frequency (pitch) ranging from 85 Hz to 180 Hz. The spectrogram will show energy concentrations (formants) at these lower frequencies. In addition, in audio and acoustics, 0 dB is typically used as a reference point representing the maximum or standard level of sound intensity. The negative dB values indicate levels that are below the reference level. For example, -10 dB means the sound intensity is 10 dB below the reference level. It is evident that the maximum level of sound intensity occurs around 128 Hz, indicating that it represents a male voice.

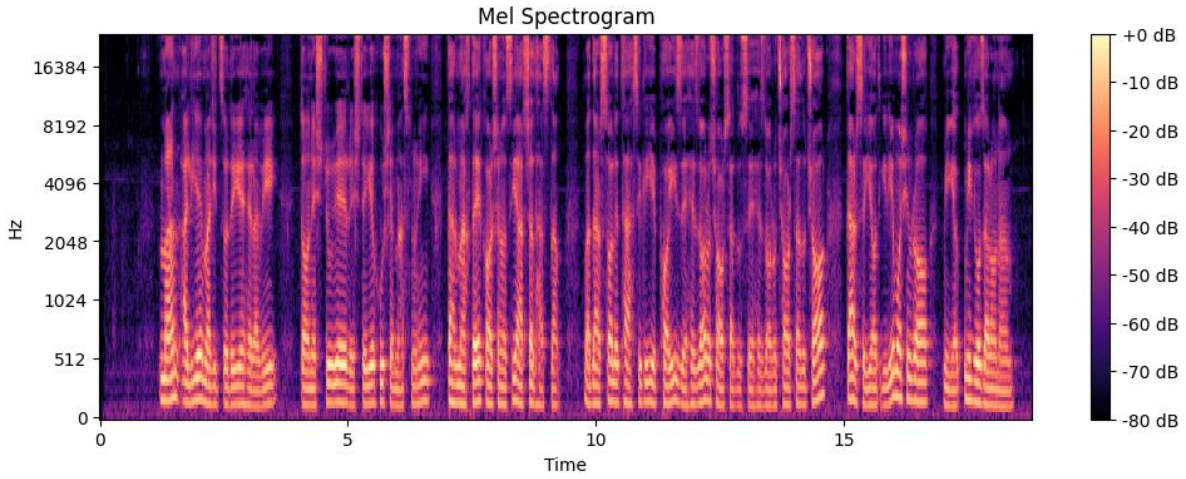


Figure 3: Mel spectrogram of a male audio sample

This Mel spectrogram depicts the frequency characteristics of a male voice over time, with strong energy concentrated in the lower frequency range, typical of male speech. The presence of well-defined harmonics and formant structures suggests voiced speech, likely containing vowels. The periodic patterns indicate natural speech dynamics, with variations in intensity reflecting different phonemes and pauses. The color gradient represents amplitude changes, with brighter areas corresponding to higher energy. Overall, the spectrogram confirms the fundamental and harmonic structures characteristic of a male voice.

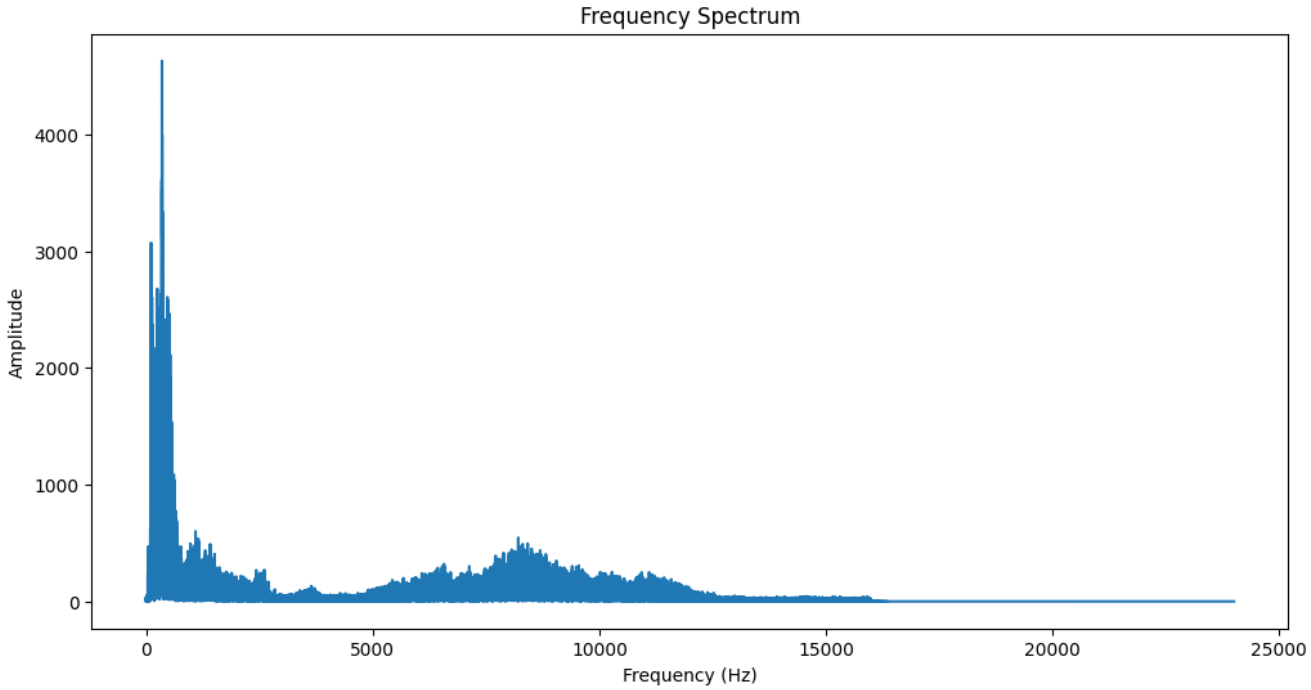


Figure 4: Fourier transform of a male audio sample

The frequency spectrum of the male voice shows a strong concentration of energy. The highest amplitude is concentrated in the **low-frequency range (below 500 Hz)**, which aligns with the fundamental frequency (F0) of a typical male voice. However, there is also noticeable noise present in the voice. This noise is likely distributed across the higher frequency regions, particularly beyond **5 kHz**, where the amplitude does not follow the expected harmonic pattern and appears more irregular. Additionally, some lower-frequency noise may be present as unintended low-amplitude fluctuations. These noisy components could result from background interference, recording artifacts, or breathiness in the speech signal.

Resampling voices

All data must be sampled at the same sampling rate to be properly de-noised and further features extracted. First, extract the sample rate of each voice:

```
for index, data in metadata.iterrows():
    path = data['path']
    y, sr = librosa.load(path, sr=None)
    metadata.at[index, 'sample_rate'] = sr
metadata['sample_rate'].value_counts()
sample_rate
48000.0    497
44100.0    315
16000.0      9
24000.0      1
```

So, there are 4 sample rates in our dataset, which must be equal. Choose 44100 Hz as the standard rate:

```
TARGET_SAMPLE_RATE = 44100
def resample_audio(input_path, output_path, target_sr=TARGET_SAMPLE_RATE)
    if os.path.exists(output_path):

        print(f"Already exists: {output_path}")
        return
    sr = metadata[metadata['path'] == input_path]['sample_rate'].values[0]
    if sr != target_sr:
        y, sr = librosa.load(input_path, sr=None)
        y_resampled = librosa.resample(y, orig_sr=sr, target_sr=target_sr)
        sf.write(output_path, y_resampled, target_sr)  # Save resampled file
        print(f"Resampled: {input_path} -> {output_path}")
    else:
        shutil.copy(input_path, output_path)
        print(f"Already at target sample rate: {input_path}")
```

Using Librosa library resample data to get the target sample rate and then save it.

```
metadata['sample_rate_resampled'].value_counts()
sample_rate_resampled
44100.0    822
```

Denoising voices

Using advanced spectral subtraction and band pass filter, do the de-noising process.

```
denoised_audio = nr.reduce_noise(y=y, sr=sr, stationary=False)
```

The `noisereduce` library uses a combination of spectral gating and deep learning-based noise reduction techniques, which are inspired by techniques like Wiener filtering and minimum statistics noise estimation. This function reduces noise by estimating the noise profile from quieter parts of the signal and applying spectral gating, which lowers the magnitude of noise-dominated frequencies. When `stationary=False`, it adaptively tracks changing noise, making it more effective for real-world scenarios. This method is more advanced than simple spectral subtraction, as it minimizes artifacts and better preserves the speech signal.

```
def butter_bandpass(lowcut, highcut, sr, order=5):
    nyquist = 0.5 * sr
    low = lowcut / nyquist
    high = highcut / nyquist
    b, a = butter(order, [low, high], btype="band")
    return b, a
def apply_bandpass(y, sr, lowcut=50, highcut=17000):
    """Filters out frequencies outside human speech range."""
    b, a = butter_bandpass(lowcut, highcut, sr)
    return filtfilt(b, a, y)
```

Defines a Butterworth band pass filter to retain frequencies within a specified range while filtering out unwanted ones. The `butter_bandpass` function calculates filter coefficients using the Butterworth design, normalizing the cutoff frequencies relative to the Nyquist frequency. The `apply_bandpass` function applies this filter using `filtfilt`, which performs zero-phase filtering to prevent signal distortion. It filters frequencies outside the human speech range (50 Hz to 17 kHz).

A voice sample after noise reduction:

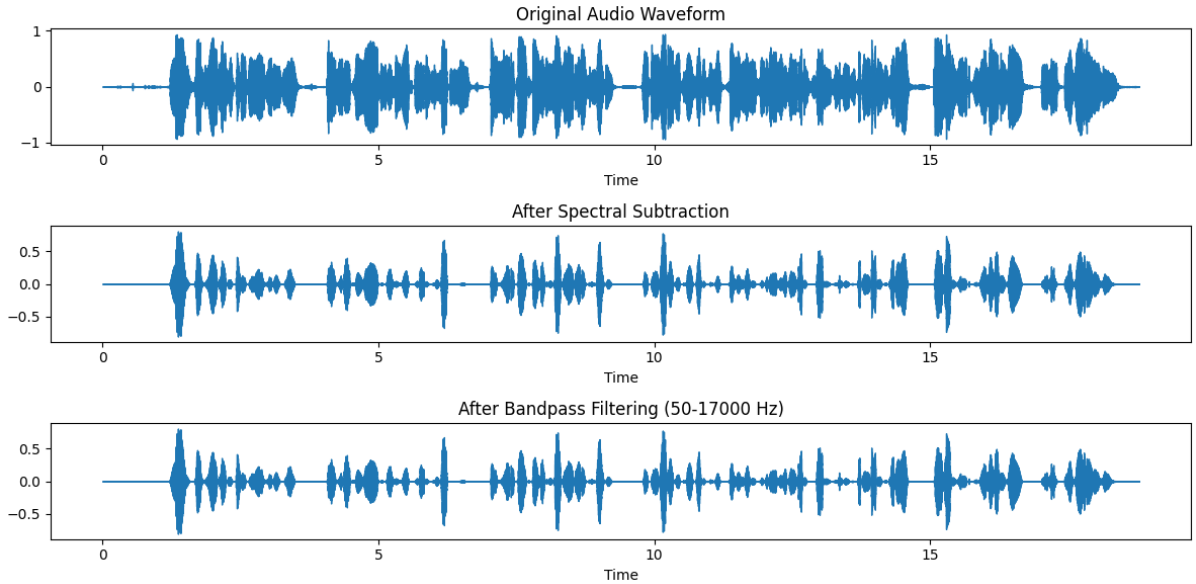


Figure 5: Voice wave from after denoising

After applying spectral subtraction, a large part of the noise is removed and we will have an almost clean and smooth sound. Applying a band pass filter will also have an effect in reducing noise, but because we don't want to lose important information, the bandwidth of the filter is wide and will not have noticeable effect.

To increase the speed of processing use **multithreading** to improve efficiency. Using `concurrent.futures` (ThreadPoolExecutor), and `tqdm` to apply noise reduction efficiently.

```
def process_all_audio():
    audio_files = [f for f in os.listdir(INPUT_DIRECTORY) if f.endswith(".mp3")]
    # Use ThreadPoolExecutor for I/O-bound tasks
    with ThreadPoolExecutor(max_workers=8) as executor:
        future_to_file = {executor.submit(denoise_audio, file): file for file in audio_files}
        # Track progress with tqdm
        for future in tqdm(as_completed(future_to_file), total=len(audio_files)):
            print(future.result())
```

Frame Segmentation (Windowing)

The windowing process is crucial in audio analysis because most signals, especially speech, are non-stationary means their frequency content changes over time. Instead of analyzing the entire signal at once, we break it into small overlapping frames to capture short-time variations while maintaining temporal continuity.

First, we do this process for a sample:

```
SAMPLE_RATE = 44100  # Standard sample rate
WINDOW_SIZE = int(SAMPLE_RATE * 0.1)  # 100 ms window
HOP_LENGTH = WINDOW_SIZE // 2  # 50% overlap
file_path = metadata['path'][0]
y, sr = librosa.load(file_path, sr=None)
y = y / np.max(np.abs(y))
frames = librosa.util.frame(y, frame_length=WINDOW_SIZE, hop_length=HOP_LENGTH)
frames = frames.T  # Transpose to have shape (n_frames, frame_length)
file_name = "test.npy"
np.save(file_name, frames)
```

Set a `sample rate` of 44.1 kHz and defines a `window size` of 100 ms (4,410 samples) with a 50% overlap (2,205 samples). The script loads an audio file from `metadata['path'][0]` using `librosa`, normalizes it, and splits it into frames using `librosa.util.frame()`. Finally, it saves the framed audio as a NumPy array (`test.npy`), which can be used for our tasks.

Now have to use a threshold to select some of these windows so that they are not in the silent section.

```
frames = np.load("test.npy")    # Shape: (num_frames, window_size)
ENERGY_THRESHOLD = 100
NUM_SELECTED_WINDOWS = 10  # Keep only 10 important frames per file
frame_energy = np.array([np.sum(frame2) for frame in frames])
voiced_frames = frames[frame_energy > ENERGY_THRESHOLD]
print(voiced_frames.shape)
if len(voiced_frames) > NUM_SELECTED_WINDOWS:
    indices = np.linspace(0, len(voiced_frames) - 1, NUM_SELECTED_WINDOWS, dtype=int)
    voiced_frames = voiced_frames[indices]
np.save("test.npy", voiced_frames)
print("Selected frames shape:", voiced_frames.shape)
```

Select the most important speech frames from an audio file by analyzing frame energy. It first loads preprocessed frames and calculates their energy levels. Frames with energy above a set threshold are retained, filtering out low-energy (unvoiced) parts. If too many voiced frames remain, it evenly selects 10 key frames to maintain efficiency. Finally, the selected frames are saved, reducing data size while keeping essential speech information for further processing.

We do this process for all data sets:

```
INPUT_DIRECTORY = "../dataset/HW1_M_cleaned"

OUTPUT_DIRECTORY = "../dataset/HW1_M_windows"

def framing_and_normalizing(file_name):

    input_path = os.path.join(INPUT_DIRECTORY, file_name)

    output_path = os.path.join(OUTPUT_DIRECTORY, file_name.replace(".mp3", ".npy"))

    if os.path.exists(output_path):

        frames = np.load(output_path)

        if frames.shape[0] >= NUM_SELECTED_WINDOWS:

            return f"Skipped: {file_name}"

    try:

        y, sr = librosa.load(input_path, sr=None)

        y = y / np.max(np.abs(y))

        frames = librosa.util.frame(y, frame_length=WINDOW_SIZE, hop_length=HOP_LENGTH)

        frames = frames.T  # Transpose to have shape (n_frames, frame_lenght)

                frame_energy = np.array([np.sum(frame2) for frame in frames])

        voiced_frames = frames[frame_energy > ENERGY_THRESHOLD] # Select frames where energy is
above a threshold (i.e., contains speech)

        energy = ENERGY_THRESHOLD

        while(len(voiced_frames) < 10):

            energy -= 10

            voiced_frames = frames[frame_energy > energy]

            if energy < 0:

                raise Exception("No voiced frames found")

        # If too many frames remain, select NUM_SELECTED_WINDOWS evenly

        if len(voiced_frames) > NUM_SELECTED_WINDOWS:

            indices = np.linspace(0, len(voiced_frames) - 1, NUM_SELECTED_WINDOWS, dtype=int)
```

```
            voiced_frames = voiced_frames[indices]
        np.save(output_path, voiced_frames)
        return f"Processed: {file_name} ({len(voiced_frames)} frames kept)"
    except Exception as e:
        return f"Error processing {file_name}: {e}"
def process_all_files():
    audio_files = [f for f in os.listdir(INPUT_DIRECTORY) if f.endswith(".mp3")]
    for file in tqdm(audio_files):
        print(framing_and_normalizing(file))
if __name__ == "__main__":
    process_all_files()
```

There is a different between here that if the number of voiced frames is too low, it gradually decreases the threshold until enough frames are found or throws an error if no voiced frames remain. If too many frames exist, it evenly selects `NUM_SELECTED_WINDOWS` frames using `np.linspace()`.

There is a voice in our data set that is empty so according to threshold no frames are extracted and we have to remove it. The voice is `HW1_Q3_810199319.mp3`.

```
metadata = metadata[metadata['path'] != '..\dataset\HW1_M\HW1_Q3_810199319.mp3']
```

feature extraction

Extract the following features from the dataset:

1. Log Mel Spectrogram

The log mel spectrogram is a time-frequency representation of an audio signal that captures how its energy is distributed across different frequency bands on the Mel scale. The Mel scale is designed to reflect how humans perceive sound, emphasizing lower frequencies where speech information is most significant. By applying a logarithmic transformation, variations in amplitude become more perceptible, making this feature particularly useful in tasks such as speech recognition, speaker identification, and music classification.

```
def compute_log_mel_spectrogram(y, sr, n_mels=23, fmin=0, fmax=None):
    # Compute Mel spectrogram
    mel_spectrogram = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=n_mels, fmin=fmin,
fmax=fmax)
    # Convert to log scale
    log_mel_spectrogram = librosa.power_to_db(mel_spectrogram)
    return log_mel_spectrogram
```

2. MFCC (Mel Frequency Cepstral Coefficients)

MFCCs represent the short-term power spectrum of an audio signal and are among the most widely used features in speech and audio processing. They are derived from the log mel spectrogram through a discrete cosine transform (DCT), which helps remove correlations between frequency bands. Since

MFCCs effectively capture the timbre and phonetic structure of speech, they are commonly applied in speech recognition, speaker verification, and emotion detection.

```
def compute_mfcc(y, sr, n_mfcc=13):
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc)
    return mfcc
```

3. Spectral Centroid

The spectral centroid represents the "center of mass" of the frequency spectrum. It is computed as a weighted mean of the frequencies present in a signal, with higher amplitude frequencies contributing more to the calculation. A higher spectral centroid indicates a brighter or sharper sound, such as a violin, whereas a lower centroid corresponds to bass-heavy sounds, such as a cello or male speech. This feature is useful in music genre classification, speech analysis, and sound texture recognition.

```
def compute_spectral_centroid(y, sr):
    spectral_centroid = librosa.feature.spectral_centroid(y=y, sr=sr)
    return spectral_centroid
```

4. Spectral Bandwidth

Spectral bandwidth measures the spread of frequencies around the spectral centroid, essentially describing how "sharp" or "wide" the frequency distribution is. A narrow bandwidth is associated with pure tones, such as a sine wave, while a broad bandwidth characterizes noise-like sounds, such as white noise or consonants like "s" and "f." This feature is commonly used in speech processing, sound classification, and music timbre analysis.

```
def compute_spectral_bandwidth(y, sr):
    spectral_bandwidth = librosa.feature.spectral_bandwidth(y=y, sr=sr)
    return spectral_bandwidth
```

5. Spectral Contrast

Spectral contrast measures the difference in amplitude between peaks and valleys within the spectrum. Unlike other spectral features that focus on averages, spectral contrast highlights the dynamic range between dominant and weaker frequencies. This makes it particularly useful for distinguishing harmonic-rich sounds, such as musical instruments, from noise-dominated sounds, like percussive elements or whispers. It is widely applied in music analysis, instrument classification, and environmental sound recognition.

```
def compute_spectral_contrast(y, sr):
    spectral_contrast = librosa.feature.spectral_contrast(y=y, sr=sr)
    return spectral_contrast
```

6. Zero-Crossing Rate

The zero-crossing rate (ZCR) calculates how often an audio signal crosses the zero amplitude axis, switching from positive to negative or vice versa. A high ZCR is typically found in noisy or percussive sounds, such as unvoiced consonants ("s" and "f"), while a low ZCR is characteristic of smooth, tonal sounds, such as vowels and musical notes. This feature is widely used in speech segmentation, music classification, and onset detection.

```
def compute_zero_crossing_rate(y):
    zero_crossing_rate = librosa.feature.zero_crossing_rate(y)
    return zero_crossing_rate
```

7. Energy

Energy is computed as the sum of squared amplitude values over a given window. It represents the loudness of the signal and is useful for detecting speech activity, distinguishing silence from voiced segments, and identifying changes in intensity over time. High-energy regions often correspond to vowels and voiced phonemes, while low-energy areas might indicate pauses or unvoiced consonants.

```
def compute_energy(y):
    energy = np.sum(np.square(y))
    return energy
```

Combining Time and Frequency Features

For a comprehensive analysis, both time-domain and frequency-domain features can be combined to create a rich feature set. Time-domain features, such as energy and zero-crossing rate, capture temporal variations in the signal, while frequency-domain features, like MFCCs and spectral centroid, provide insights into tonal and timbral characteristics. By integrating both types of features, models can be developed for applications such as speech recognition, speaker identification, emotion detection, and music classification.

Feature Extraction for Each Window

Now that we have a function to extract features for each window, we can iterate over each window and apply the feature extraction.

The important point is that each of these functions divides each window into several sub-windows and calculates the features for it, which causes the number of features to increase. To avoid this, we consider each feature as the average feature extracted from these sub-windows.

```
def extract_features(y, sr):
    """Extracts features from an entire waveform (concatenated frames)."""
    # Zero Crossing Rate
    zero_crossing_rate = librosa.feature.zero_crossing_rate(y).mean()
    # Energy (Root Mean Square)
    energy = librosa.feature.rms(y=y).mean()
    # Log Mel Spectrogram (23 Mel bands)
    log_mel_spectrogram = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=23)
    log_mel_spectrogram = librosa.power_to_db(log_mel_spectrogram,ref=np.max).mean(axis=1)
    # MFCCs (13 coefficients)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13).mean(axis=1)
    # Spectral Centroid
    spectral_centroid = librosa.feature.spectral_centroid(y=y, sr=sr).mean()
    # Spectral Bandwidth
    spectral_bandwidth = librosa.feature.spectral_bandwidth(y=y, sr=sr).mean()
    # Spectral Contrast (7 bands)
    spectral_contrast = librosa.feature.spectral_contrast(y=y, sr=sr).mean(axis=1)
```

```
    feature_dict = {
        "zero_crossing_rate": zero_crossing_rate,
        "energy": energy,
        "spectral_centroid": spectral_centroid,
        "spectral_bandwidth": spectral_bandwidth
    }
    for i, val in enumerate(log_mel_spectrogram):
        feature_dict[f"log_mel_{i+1}"] = val

    for i, val in enumerate(mfcc):
        feature_dict[f"mfcc_{i+1}"] = val

    for i, val in enumerate(spectral_contrast):
        feature_dict[f"spectral_contrast_{i+1}"] = val

    return feature_dict
```

This function extracts various audio features from a waveform to analyze its characteristics. It computes the zero-crossing rate (measuring signal transitions), energy (RMS) (indicating loudness), and log mel spectrogram (capturing frequency content across 23 mel bands). It also extracts MFCCs (13 coefficients representing timbre), spectral centroid (indicating the center of mass of frequencies), spectral bandwidth (measuring frequency spread), and spectral contrast (highlighting differences between spectral peaks and valleys). The extracted features are stored in a dictionary, where each feature is labeled appropriately, including numbered entries for mel bands, MFCCs, and spectral contrast values.

The following function, loads .npy file, concatenates frames, extracts features, and returns a dictionary:

```
def process_audio_file(file_path):
    try:
        frames = np.load(file_path)  # Shape: (num_windows, window_size)
        # Concatenate frames into a single waveform
        waveform = frames.flatten()  # 1D array
        sr = 44100
        features = extract_features(waveform, sr)
        file_name = os.path.basename(file_path).replace(".npy", "")
        features["file_name"] = file_name
        return features
        except Exception as e:
        print(f"Error processing {file_path}: {e}")
        return None
```

To increase the speed of processing use **multithreading** to improve efficiency.

The final result will be saved as .csv file named "extracted_features.csv".

Now we want to concatenate metadata and this extracted data set so we use the following code:

```
dataset['gender'] = ""
dataset['question_number'] = ""
dataset['student_id'] = ""
dataset['has_intro'] = ""


for index, row in dataset.iterrows():
    file_name = row['file_name']
    metadata_index = metadata[metadata['filename'].str.contains(file_name, na=False)].index
    if not metadata_index.empty:
        dataset.at[index, 'gender'] = metadata.at[metadata_index[0], 'gender']
        dataset.at[index, 'question_number'] = metadata.at[metadata_index[0], 'question_number']
        dataset.at[index, 'student_id'] = metadata.at[metadata_index[0], 'student_id']
        dataset.at[index, 'has_intro'] = metadata.at[metadata_index[0], 'has_intro']
```

So the final data set has 821 rows and the following collumns:

```
RangeIndex: 821 entries, 0 to 820
Data columns (total 52 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   zero_crossing_rate    821 non-null    float64
 1   energy                821 non-null    float64
 2   spectral_centroid     821 non-null    float64
 3   spectral_bandwidth    821 non-null    float64
 4   log_mel_1             821 non-null    float64
 5   log_mel_2             821 non-null    float64
 6   log_mel_3             821 non-null    float64
 7   log_mel_4             821 non-null    float64
 8   log_mel_5             821 non-null    float64
 9   log_mel_6             821 non-null    float64
 10  log_mel_7             821 non-null    float64
 11  log_mel_8             821 non-null    float64
 12  log_mel_9             821 non-null    float64
 13  log_mel_10            821 non-null    float64
 14  log_mel_11            821 non-null    float64
 15  log_mel_12            821 non-null    float64
 16  log_mel_13            821 non-null    float64
 17  log_mel_14            821 non-null    float64
 18  log_mel_15            821 non-null    float64
 19  log_mel_16            821 non-null    float64
 20  log_mel_17            821 non-null    float64
 21  log_mel_18            821 non-null    float64
 22  log_mel_19            821 non-null    float64
 23  log_mel_20            821 non-null    float64
 24  log_mel_21            821 non-null    float64
 25  log_mel_22            821 non-null    float64
 26  log_mel_23            821 non-null    float64
 27  mfcc_1                821 non-null    float64
 28  mfcc_2                821 non-null    float64
 29  mfcc_3                821 non-null    float64
 30  mfcc_4                821 non-null    float64
 31  mfcc_5                821 non-null    float64
 32  mfcc_6                821 non-null    float64
 33  mfcc_7                821 non-null    float64
 34  mfcc_8                821 non-null    float64
 35  mfcc_9                821 non-null    float64
 36  mfcc_10               821 non-null    float64
 37  mfcc_11               821 non-null    float64
 38  mfcc_12               821 non-null    float64
 39  mfcc_13               821 non-null    float64
 40  spectral_contrast_1   821 non-null    float64
 41  spectral_contrast_2   821 non-null    float64
 42  spectral_contrast_3   821 non-null    float64
 43  spectral_contrast_4   821 non-null    float64
 44  spectral_contrast_5   821 non-null    float64
 45  spectral_contrast_6   821 non-null    float64
 46  spectral_contrast_7   821 non-null    float64
 47  file_name             821 non-null    object
 48  gender                821 non-null    object
 49  question_number       821 non-null    object
 50  student_id            821 non-null    object
 51  has_intro             821 non-null    object
```

In the end, we saved the dataset as a .csv file named: "final_dataset.csv"

Visualization

In this section, we show the visual representation of the data.
First, we draw the histogram of four features `'zero_crossing_rate'`, `'energy'`, `'spectral_centroid'`, `'spectral_bandwidth'` to see the distribution of their dispersion .

```
features_to_compare = ['zero_crossing_rate', 'energy', 'spectral_centroid',
'spectral_bandwidth']
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(14, 10))
axes = axes.flatten()
for i, feature in enumerate(features_to_compare):
    axes[i].hist(data_male[feature], bins=30, alpha=0.5, label='Male', color='blue')
    axes[i].hist(data_female[feature], bins=30, alpha=0.5, label='Female', color='red')
    axes[i].set_title(f'Histogram of {feature}')
    axes[i].set_xlabel(feature)
    axes[i].set_ylabel('Frequency')
    axes[i].legend()
plt.tight_layout()
plt.show()
```

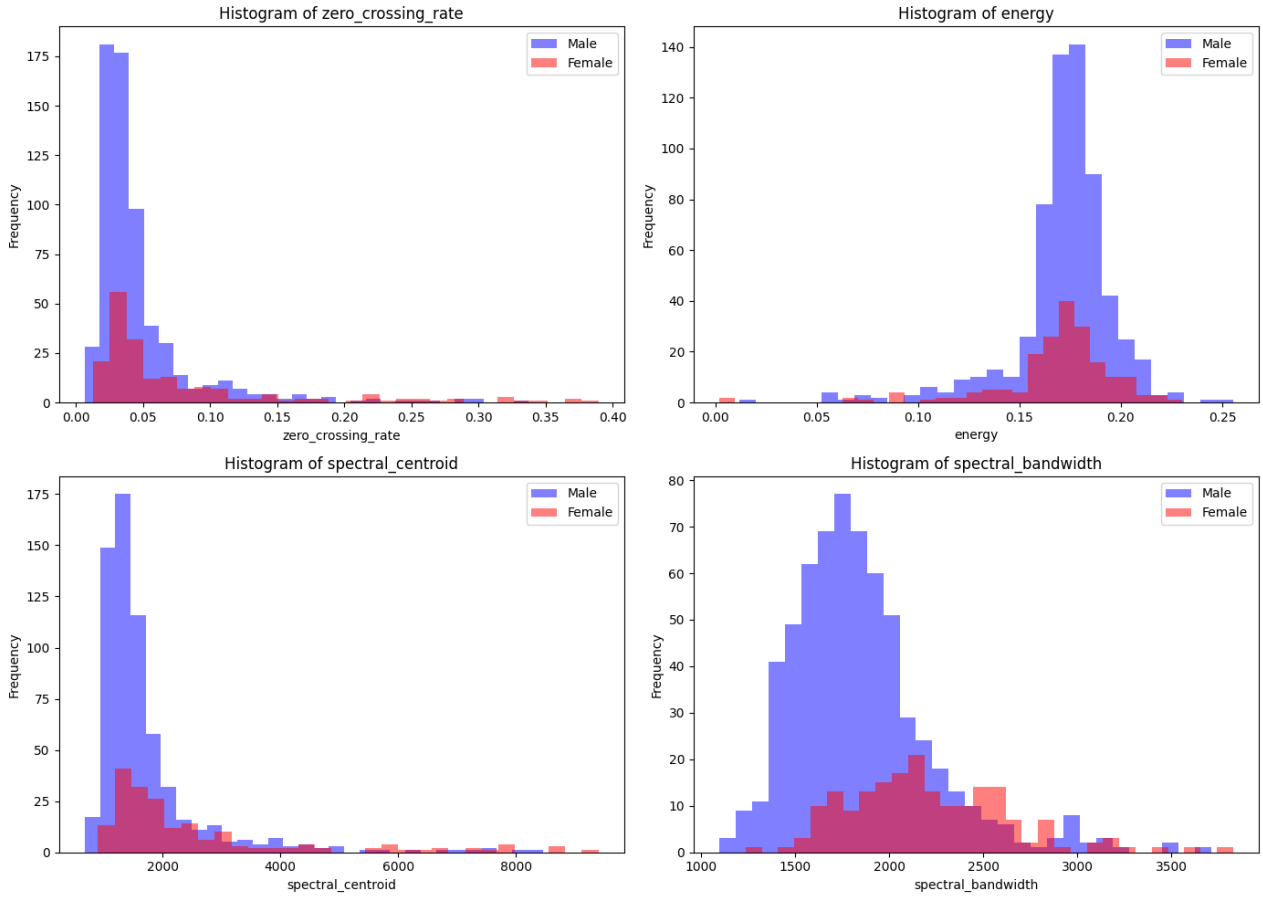


Figure 6: Histogram of features

Based on this figure male samples dominate in all four histograms. Female samples tend to appear more frequently at slightly higher values of spectral centroid and spectral bandwidth, and there is significant overlap in distributions, but male and female speech characteristics might still show some differences based on these features.

Bar plot for average of features in dataset:

```
features_to_compare = ['zero_crossing_rate', 'energy', 'spectral_centroid',
'spectral_bandwidth']
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(14, 10))
axes = axes.flatten()
for i, feature in enumerate(features_to_compare):
    male = data_male[feature].mean()
    female = data_female[feature].mean()
    axes[i].bar(['Male', 'Female'], [male, female], color=['blue', 'red'])
    axes[i].set_title(f'Average {feature} by Gender')
    axes[i].set_ylabel(feature)
plt.tight_layout()
plt.show()
```

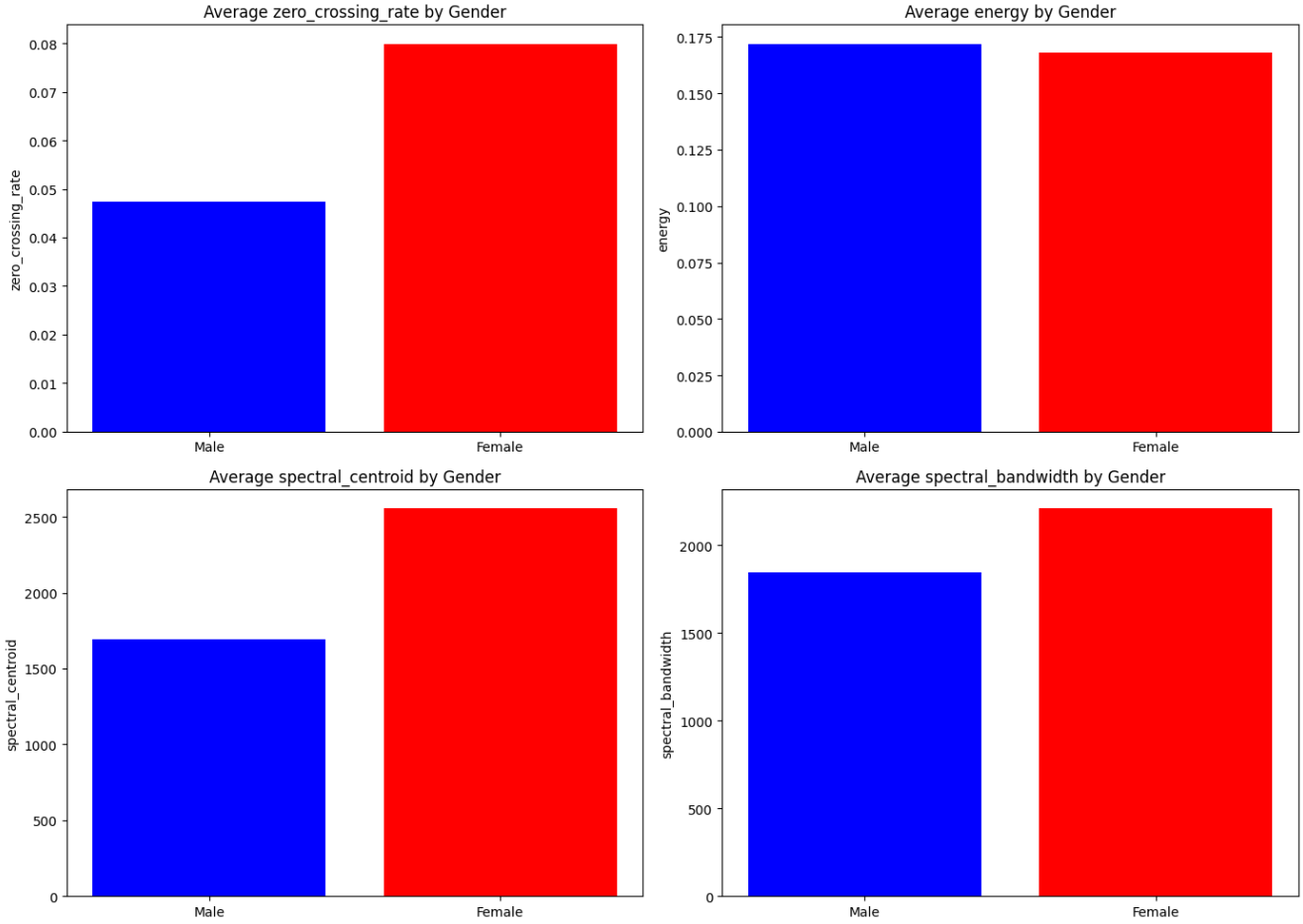


Figure 7: Bar plot of features

Based on this plot, `'zero_crossing_rate'` and `'spectral_centroid'`are good features for separability criterion and help in discrimination, but `'energy'` and `'spectral_bandwidth'` are too close and maybe make discrimination difficult.

Also we can plot the feature based on each other to check their correlation.

```
features_to_compare = ['zero_crossing_rate', 'energy', 'spectral_centroid',
'spectral_bandwidth']
sns.pairplot(dataset, vars=features_to_compare, hue='gender', palette='coolwarm')
plt.show()
```

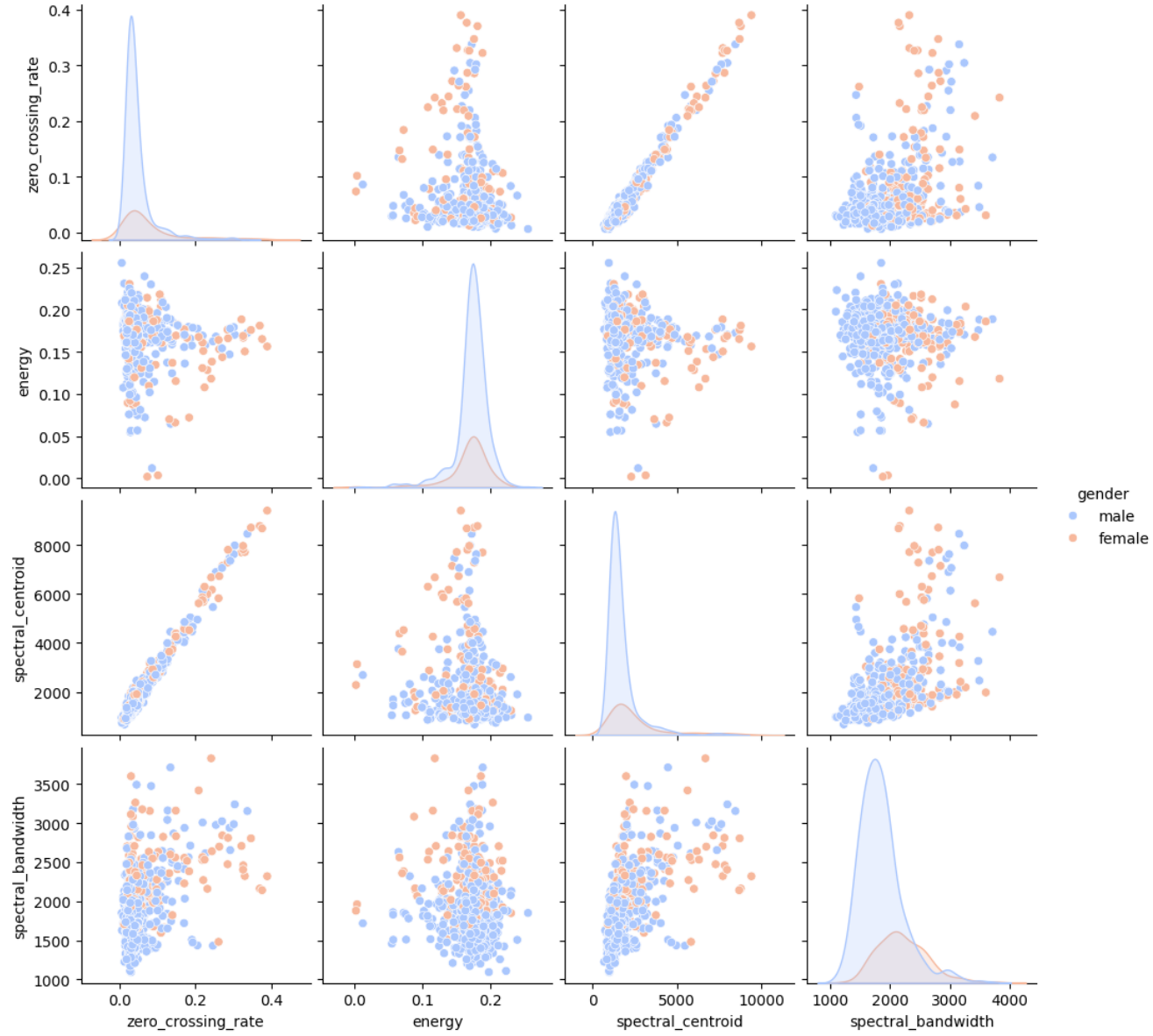


Figure 8: pairplot of features

The pairplot reveals gender-based differences in speech features. Male samples (blue) dominate lower values in zero crossing rate, spectral centroid, and spectral bandwidth, indicating lower-frequency speech characteristics. Female samples (orange) are more prevalent at higher values, suggesting richer high-frequency content. Energy distribution shows males clustering around specific values, while females have a broader spread. Feature correlations indicate strong relationships, especially between zero crossing rate and spectral centroid, with female voices tending towards higher values. While overlap exists, these trends suggest potential for gender classification based on these acoustic features.

We apply PCA transform for sketch the plot of features:

```
features = dataset.columns[:-5]
scaler = StandardScaler()
scaled_features = scaler.fit_transform(dataset[features])

pca = PCA(n_components=3)
pca_result = pca.fit_transform(scaled_features)

dataset['PCA1'] = pca_result[:, 0]
dataset['PCA2'] = pca_result[:, 1]
dataset['PCA3'] = pca_result[:, 2]
```

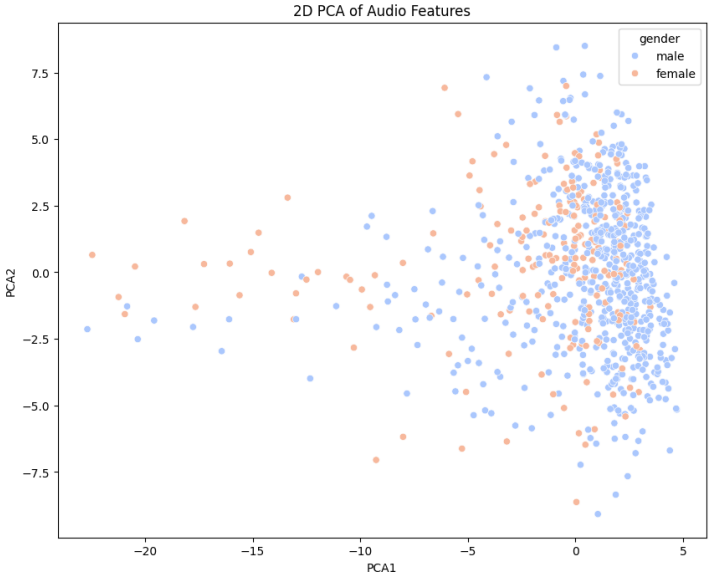


Figure 9: 2D PCA plot of audio features

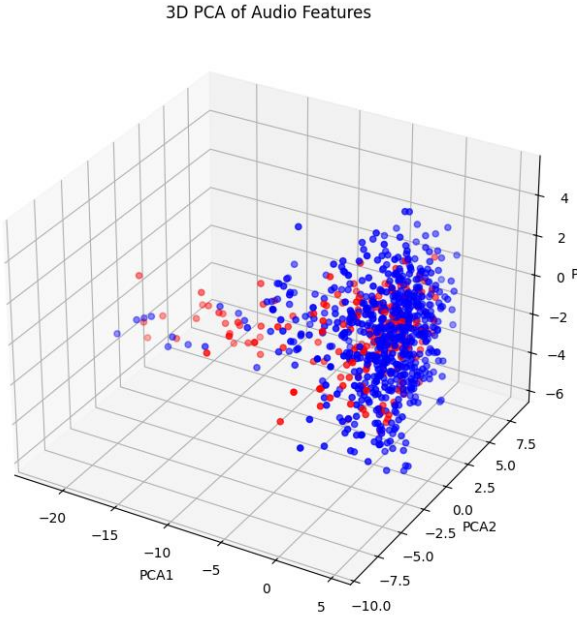


Figure 10: 3D PCA plot of audio features

Classification

In this part, the goal is to classify using the extracted features from the last part. We have two main goals, first we want to detect the gender of the speaker and then we want to recognize the identity of the speaker, and this part is done in closed-way authentication way.

Gender Classification

We must use at least 25 % of the whole data for this phase.

First, we get the value counts of gender column to find out how many data rows we must use to train our models. The result is:

```
gender
male      631
female    190
```

Then we map the gender column and encode it to numerical values to use it in out models:

```
dataset['gender'] = dataset['gender'].apply(lambda x: 1 if x == 'male' else 0)
```

We drop some columns that doesn't provide any information in our model:

```
gender_dataset = dataset.drop(columns=['has_intro', 'student_id', 'question_number',
'file_name'], inplace=False)
```

After that, we want to select data from our dataset to have equal samples per each gender. So, after that we will have a dataset which has 190 samples for each gender. (totally 380 samples)

```
male_samples = gender_dataset[gender_dataset['gender'] == 1].sample(n=190, random_state=42)
female_samples = gender_dataset[gender_dataset['gender'] == 0].sample(n=190,
random_state=42)
balanced_dataset = pd.concat([male_samples, female_samples])
```

Then we check the correlation of the features. If two features are highly correlated, we have to remove one of the pair because they cause redundancy and may mislead the model. So, we define the get_corr function and by calling that, the `corr_matrix` is returned, and it prints top highly correlated features. (corr > 0.9)

```
def get_corr(data):
    num_cols = data.select_dtypes(include=['int64', 'float64']).columns
    corr_matrix = data[num_cols].corr()
    upper_triangle = corr_matrix.where(np.triu(np.ones(corr_matrix.shape),
k=1).astype(bool))
    high_corr_features = [(column, upper_triangle[column].idxmax(),
upper_triangle[column].max()) for column in upper_triangle.columns if
any(upper_triangle[column] > 0.9)]
    for feature1, feature2, corr_value in high_corr_features:
        print(f"{feature1} and {feature2} have a correlation of {corr_value}")
    return corr_matrix
```

spectral_centroid and zero_crossing_rate have a correlation of 0.9903635146873123
log_mel_5 and log_mel_4 have a correlation of 0.9460780907280261
log_mel_6 and log_mel_5 have a correlation of 0.9659725629604006
log_mel_7 and log_mel_6 have a correlation of 0.960022540388093
log_mel_8 and log_mel_7 have a correlation of 0.9538645703183916
log_mel_9 and log_mel_8 have a correlation of 0.9228917991222666
log_mel_10 and log_mel_9 have a correlation of 0.910924004730554
log_mel_12 and log_mel_11 have a correlation of 0.901269345264183
log_mel_17 and log_mel_16 have a correlation of 0.9337996433657169
log_mel_18 and log_mel_17 have a correlation of 0.9393516028918286
log_mel_19 and log_mel_18 have a correlation of 0.933225958834208
log_mel_20 and log_mel_19 have a correlation of 0.9417058169778803
log_mel_21 and log_mel_20 have a correlation of 0.9734443556833696
log_mel_22 and log_mel_21 have a correlation of 0.9671741526035247
mfcc_2 and log_mel_3 have a correlation of 0.900788847611454

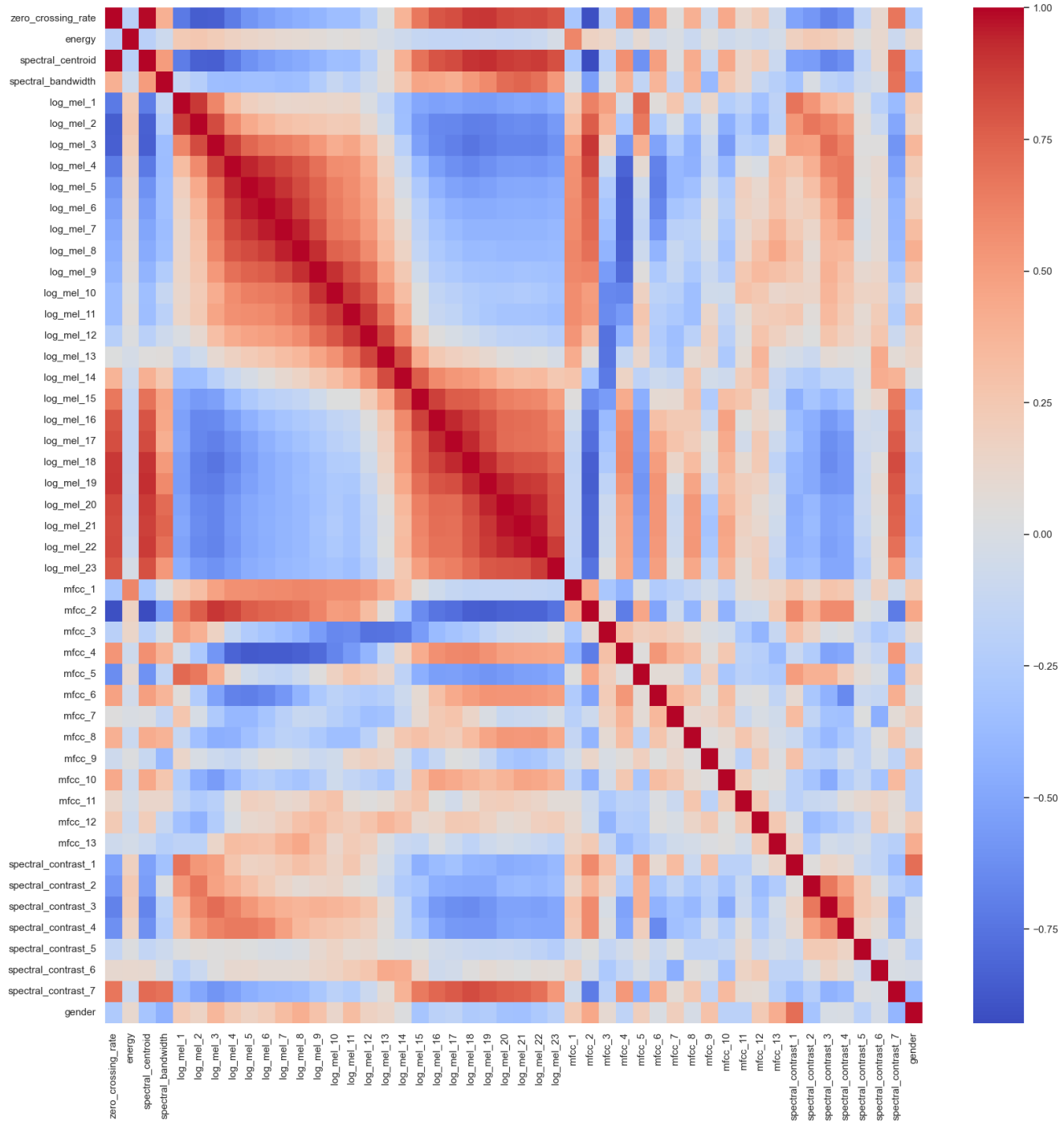


Figure 11: correlation Matrix

We can see that dataset has highly correlated log-mel spectrogram features across adjacent frequency bands. This happens because adjacent frequency bins in a spectrogram often contain similar information due to how mel filters work. In order to handle this, we applied PCA only to the log-mel features to reduce them into fewer uncorrelated components. This reduces the number of log-mel features while keeping 95% of the information.

```
df = balanced_dataset.copy()

log_mel_features = [col for col in df.columns if "log_mel" in col]
log_mel_data = df[log_mel_features]

scaler = StandardScaler()
log_mel_scaled = scaler.fit_transform(log_mel_data)

pca = PCA(n_components=0.95)
log_mel_pca = pca.fit_transform(log_mel_scaled)

pca_columns = [f"log_mel_pca_{i}" for i in range(log_mel_pca.shape[1])]
df_pca = pd.DataFrame(log_mel_pca, columns=pca_columns)

df = df.drop(columns=log_mel_features)
df.reset_index(drop=True, inplace=True)
df_pca.reset_index(drop=True, inplace=True)
df = pd.concat([df, df_pca], axis=1)

print(f"Original log-mel features: {len(log_mel_features)}, Reduced to:
{len(pca_columns)}")
```

```
Original log-mel features: 23, Reduced to: 8
```

After PCA we can see that 23 log-mel features are reduced to 8 features.

After getting the correlation again, we have decided to drop two columns:

```
spectral_centroid and zero_crossing_rate have a correlation of 0.9903635146873123
log_mel_pca_0 and mfcc_2 have a correlation of 0.9786177248941503
```

since these two features had less correlation with the target column, we choose these two to drop.

```
df.drop(columns=['zero_crossing_rate', 'log_mel_pca_0'], inplace=True)
```

Now we can see that our dataset has 30 features. Now in order to visualize the data we apply PCA over the whole dataset and we get the following plots:

```
def plot_PCA(df, target, n_components):
    scaler = StandardScaler()
    scaled_features = scaler.fit_transform(df.drop(columns=[target]))

    pca = PCA(n_components=n_components)
    pca_result = pca.fit_transform(scaled_features)

    if n_components == 2:
        df['PCA1'] = pca_result[:, 0]
        df['PCA2'] = pca_result[:, 1]

        plt.figure(figsize=(10, 8))
        sns.scatterplot(x='PCA1', y='PCA2', hue=target, data=df, palette='coolwarm')
        plt.title('2D PCA of Audio Features')
        plt.show()

        df.drop(columns=['PCA1', 'PCA2'], inplace=True)

    elif n_components == 3:
        df['PCA1'] = pca_result[:, 0]
        df['PCA2'] = pca_result[:, 1]
        df['PCA3'] = pca_result[:, 2]

        fig = plt.figure(figsize=(10, 8))
        ax = fig.add_subplot(111, projection='3d')
        colors = {1: 'blue', 0: 'red', 2: 'green', 3: 'yellow', 4: 'purple', 5: 'orange'}
        ax.scatter(df['PCA1'], df['PCA2'], df['PCA3'], c=df[target].apply(lambda x:
colors[x]))
        ax.set_xlabel('PCA1')
        ax.set_ylabel('PCA2')
        ax.set_zlabel('PCA3')
        plt.title('3D PCA of Audio Features')
        plt.show()

        df.drop(columns=['PCA1', 'PCA2', 'PCA3'], inplace=True)


plot_PCA(df, 'gender', 2)
plot_PCA(df, 'gender', 3)
```

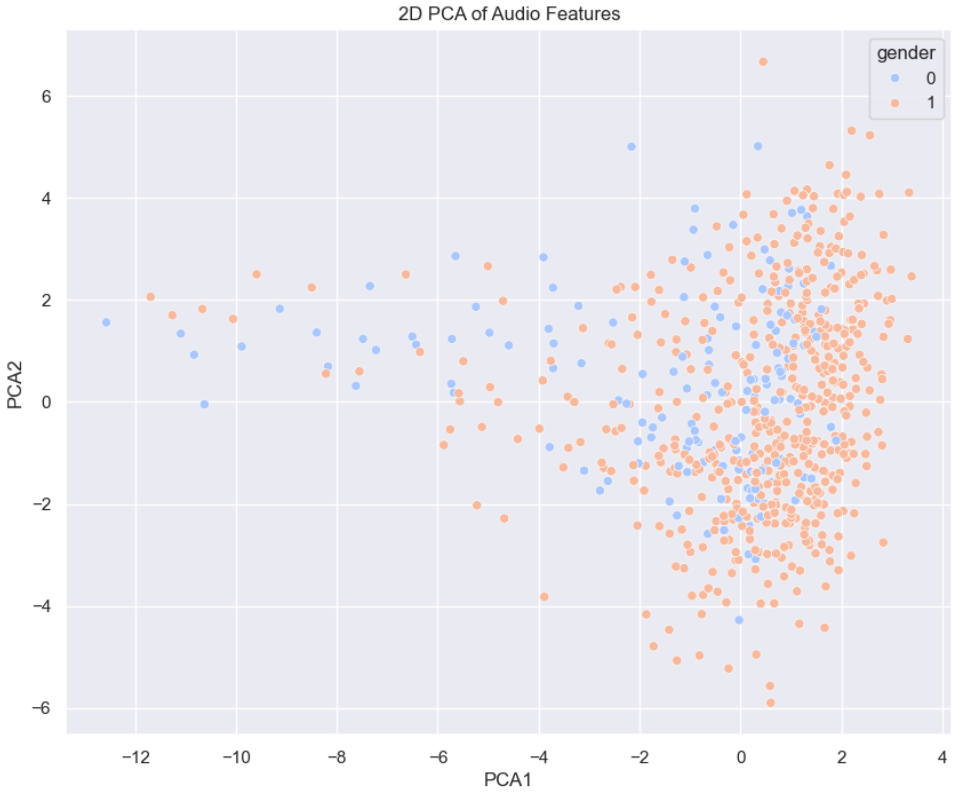


Figure 12: 2D PCA plot of audio features

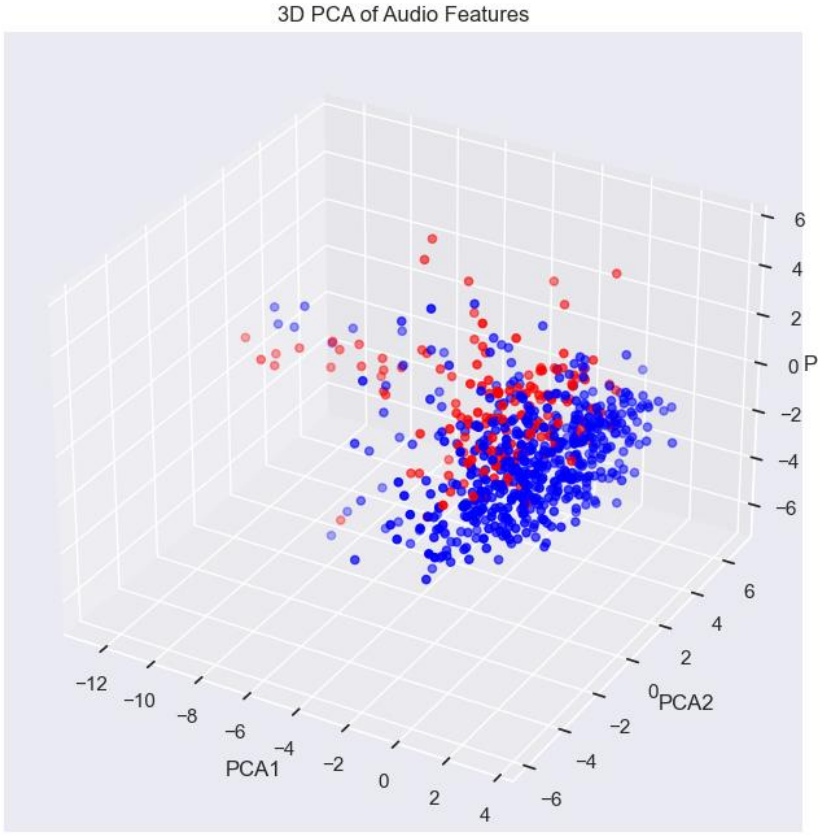


Figure 13: 3D PCA plot of features

Normalization

The final step before training the models is to split and normalize the data, so we split the data into train and test sets and then we normalize it. The test size is 95/380 to have 25 percent of data for test set and stratify = y is set to have equal number of samples for each gender.

```
X = df.drop(columns=["gender"])
y = df["gender"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=95/380,
random_state=42, stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Also, for normalizing we fit the scaler on training data and then transform the test data according to that.

Training Models

Now it is the time for defining and training the models and then evaluating them. We have defined four models.

1. MLP (Multi-Layer Perceptron)

2. SVM (Support Vector Machine)

3. Logistic Regression

4. KNN (K-Nearest Neighbors)

Each of these models is suitable for binary classification tasks, like gender classification, but they have different strengths and weaknesses.

1. MLP (Multi-Layer Perceptron) - Neural Network Approach

- MLP is a deep learning-based approach that can model complex relationships between features.
- It can learn nonlinear decision boundaries, making it more powerful than simpler models like Logistic Regression.
- Works well when the dataset is large and has meaningful hidden patterns.

- Downsides:

- Requires more training time and hyper parameter tuning (e.g., number of layers, neurons, activation functions).
- More prone to overfitting with small datasets.

2. SVM (Support Vector Machine) - Margin-Based Classifier

- Effective for high-dimensional data and works well even when features are correlated.
- Can model nonlinear decision boundaries if using a kernel (e.g., RBF kernel).
- Robust to overfitting in low-dimensional spaces.

- Downsides:

- Training time increases with large datasets.
- Choosing the right kernel is crucial for performance.

3. Logistic Regression - Simple & Interpretable Model

- A simple and effective baseline model for binary classification.
- Works well when the data is linearly separable.
- Computationally efficient and easy to interpret (coefficients indicate feature importance).

- Downsides:

- Limited to linear decision boundaries; struggles with complex patterns.
- Sensitive to outliers in the dataset.

4. KNN (K-Nearest Neighbors) - Instance-Based Learning

- Simple and intuitive: it classifies a sample based on the majority class of its nearest neighbors.
- No assumptions about the underlying data distribution.
- Works well when class boundaries are irregular.

- Downsides:

- Slower for large datasets because it requires computing distances for all points.
- Performance depends on choosing an appropriate k value.

- If the data is linearly separable, Logistic Regression and SVM (with a linear kernel) are great choices.

- If the data is nonlinear, MLP or SVM (with an RBF kernel) are better.

- If dataset is small, KNN can be effective.

- If we want powerful and deep learning approach, MLP is the way to go.

```
models = {
    "SVM": SVC(kernel="rbf", probability=True, random_state=42),
    "Logistic Regression": LogisticRegression(random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5)
}
predictions = {
    "MLP": [],
    "SVM": [],
    "Logistic Regression": [],
    "KNN": [],
}
probabilities = {
    "MLP": [],
    "SVM": [],
```

```
    "Logistic Regression": [],
    "KNN": [],
}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]
    predictions[name] = y_pred
    probabilities[name] = y_prob
    accuracy = accuracy_score(y_test, y_pred)
    print(f" {name} Accuracy: {accuracy:.4f}")
MLP = MLPClassifier(
    hidden_layer_sizes=(128, 64, 32),
    activation="relu",
    solver="adam",
    alpha=0.02,
    batch_size=16,
    learning_rate="adaptive",
    max_iter=1000,
    random_state=42
)
MLP.fit(X_train, y_train)
y_pred_optimized = MLP.predict(X_test)
y_proba = MLP.predict_proba(X_test)[:, 1]
predictions["MLP"] = y_pred_optimized
probabilities["MLP"] = y_proba
accuracy_optimized = accuracy_score(y_test, y_pred_optimized)
print(f"MLP Accuracy: {accuracy_optimized:.4f}")
print(classification_report(y_test, y_pred_optimized))
```

Results:

```
SVM Accuracy: 0.9598
              precision    recall  f1-score   support

           0       0.97      0.85      0.90        39
           1       0.96      0.99      0.97       135

    accuracy                           0.96       174
   macro avg       0.96      0.92      0.94       174
weighted avg       0.96      0.96      0.96       174

 Logistic Regression Accuracy: 0.9540
              precision    recall  f1-score   support

           0       0.94      0.85      0.89        39
           1       0.96      0.99      0.97       135
```

```
    accuracy                           0.95       174
   macro avg       0.95      0.92      0.93       174
weighted avg       0.95      0.95      0.95       174

 KNN Accuracy: 0.9483
              precision    recall  f1-score   support

           0       0.92      0.85      0.88        39
           1       0.96      0.98      0.97       135

    accuracy                           0.95       174
   macro avg       0.94      0.91      0.92       174
weighted avg       0.95      0.95      0.95       174
MLP Accuracy: 0.9684
              precision    recall  f1-score   support

           0       0.94      1.00      0.97        48
           1       1.00      0.94      0.97        47

    accuracy                           0.97        95
   macro avg       0.97      0.97      0.97        95
weighted avg       0.97      0.97      0.97        95
```

We also defined two functions to plot ROC curve and confusion matrix:

```
def ROC_curve(y_true, y_prob, model_name):
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    roc_auc = auc(fpr, tpr)

    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC curve (AUC = {roc_auc:.2f})')
    plt.plot([0, 1], [0, 1], color='gray', linestyle='--')  # Diagonal line
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(model_name +' ROC Curve')
    plt.legend(loc="lower right")
    plt.grid()
    plt.show()
def confusion_matrix_plot(y_true, y_pred, model_name):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Male", "Female"],
yticklabels=["Male", "Female"])
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title(model_name+ " Confusion Matrix")
    plt.show()
```

The result of each model

Result of MLP model:

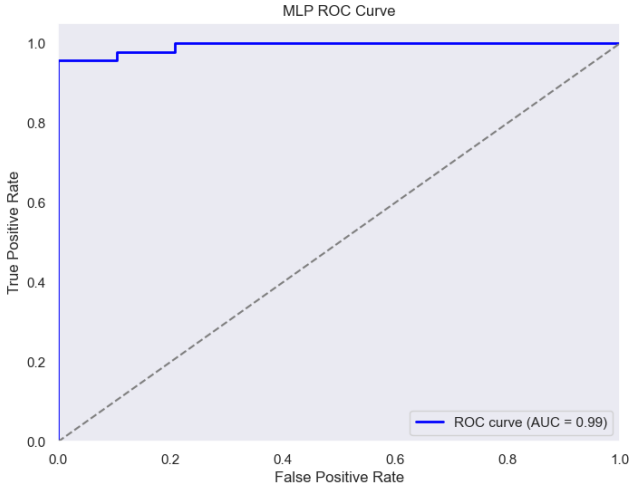


Figure 14: MLP ROC curve in Gender Classification

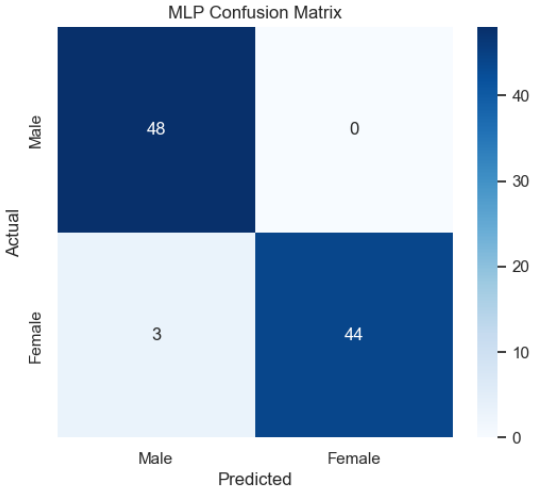


Figure 15: MLP confusion matrix in Gender Classification

From the confusion matrix we can see that 3 samples of female voices had been misclassified as male. The AUC is 0.99 which indicates that our model is a good classifier.

Result of SVM model:

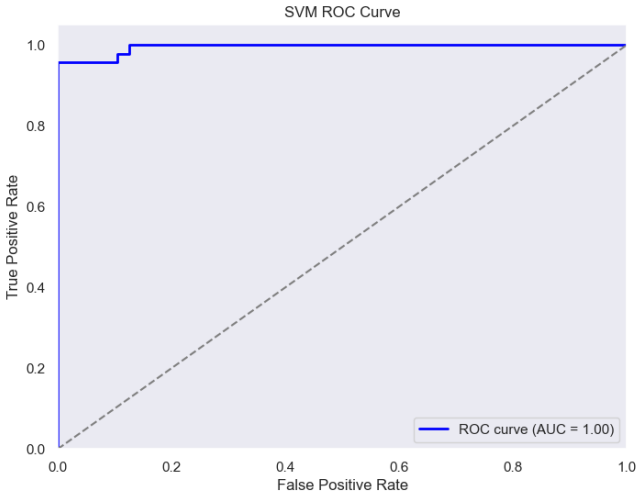


Figure 17: SVM ROC curve in Gender Classification

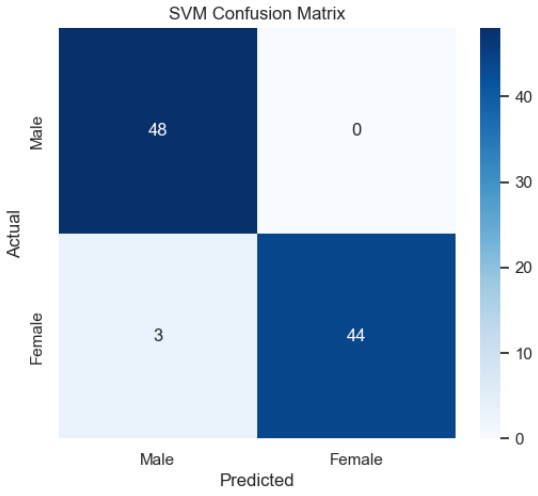


Figure 16: SVM confusion matrix in Gender Classification

The result of SVM model is as same as MLP model.

Result of Logistic Regression model:

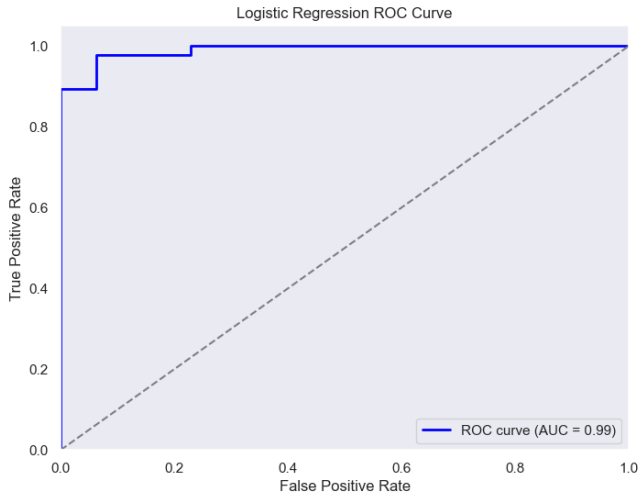


Figure 18: Logistic Regression ROC curve in Gender Classification

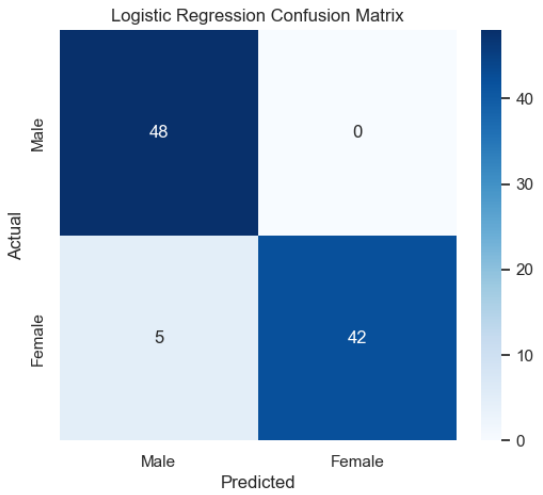


Figure 19: Logistic Regression confusion matrix in Gender Classification

Result of KNN model:

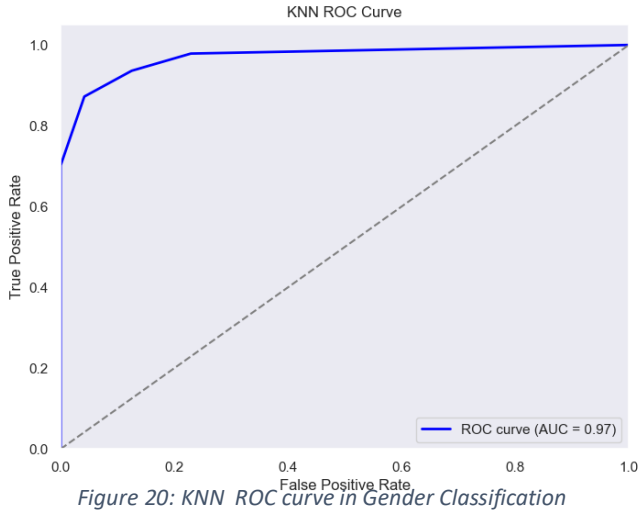


Figure 20: KNN ROC curve in Gender Classification

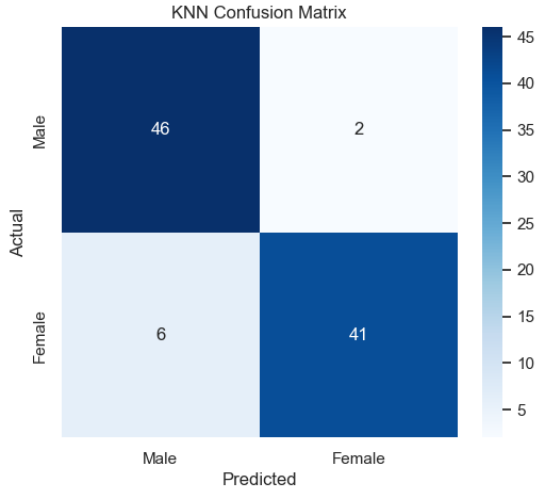


Figure 21: KNN confusion matrix in Gender Classification

The

SVM and MLP models both achieved the highest accuracy of 0.9684, indicating that they performed similarly well in classifying the data. Both of these models are capable of capturing complex patterns in the data, with SVM being effective in high-dimensional spaces and MLP potentially benefiting from its deep architecture. Logistic Regression, with an accuracy of 0.9474, performed slightly worse, which is expected since it is a linear model that may struggle with more complex patterns in data. KNN, with an accuracy of 0.9158, had the lowest performance, which is common for this model when dealing with large datasets or high-dimensional features, as it relies on the distance metric, which can become less effective in such scenarios. Overall, SVM and MLP appear to be the most suitable models for this particular problem, while Logistic Regression and KNN offer less robust results.

Identity Recognition (Closed-Set Authentication)

First of all, we check the value counts of student_ID column to select the number of samples for traing our model. We can see that most of the values are equal to 7, So after that we drop the student IDs that have less than 7 samples and if we have more than7 from one student we drop some of then to have 7 samples.

```
# Filter out student_ids with less than 7 occurrences
student_counts = dataset['student_id'].value_counts()
students_to_keep = student_counts[student_counts >= 7].index
filtered_dataset = dataset[dataset['student_id'].isin(students_to_keep)]
# Ensure no student_id has more than 7 occurrences
for student_id in students_to_keep:
    student_data = filtered_dataset[filtered_dataset['student_id'] == student_id]
    if len(student_data) > 7:
        filtered_dataset = filtered_dataset.drop(student_data.index[7:])
```

Same as last phase, we drop some irrelavant columns.

```
filtered_dataset.drop(columns=['has_intro', 'question_number', 'file_name'], inplace=True)
```

Again we calculate the correlation of features and same as the last phase we used PCA to reduce correlation and dimension.

```
Original log-mel features: 23, Reduced to: 8
```

And we drop two columns which they were highly correlated:

```
df.drop(columns=['zero_crossing_rate', 'log_mel_pca_0'], inplace=True)
```

Now we have 31 columns.

For each set of training we use 6 student's data, using the following code and randomly. Also after that using label encoder we encode the object data to numerical so we can use it in our model:

```
def get_6_students_data():
    student_counts = dataset['student_id'].value_counts()
    students_to_keep = student_counts[student_counts == 7].index
    selected_students = np.random.choice(students_to_keep, size=6, replace=False)
    data =  df[df['student_id'].isin(selected_students)]
    le = LabelEncoder()
    data['student_id'] = le.fit_transform(data['student_id'])
    return data
data = get_6_students_data()
```

For example these student IDs are selected and we have 7 samples for each.

```
student_id
810100261    7      810103019    7      810103057    7 810103140    7
810600097    7      810600065    7
```

Now we use PCA to visualize our data:

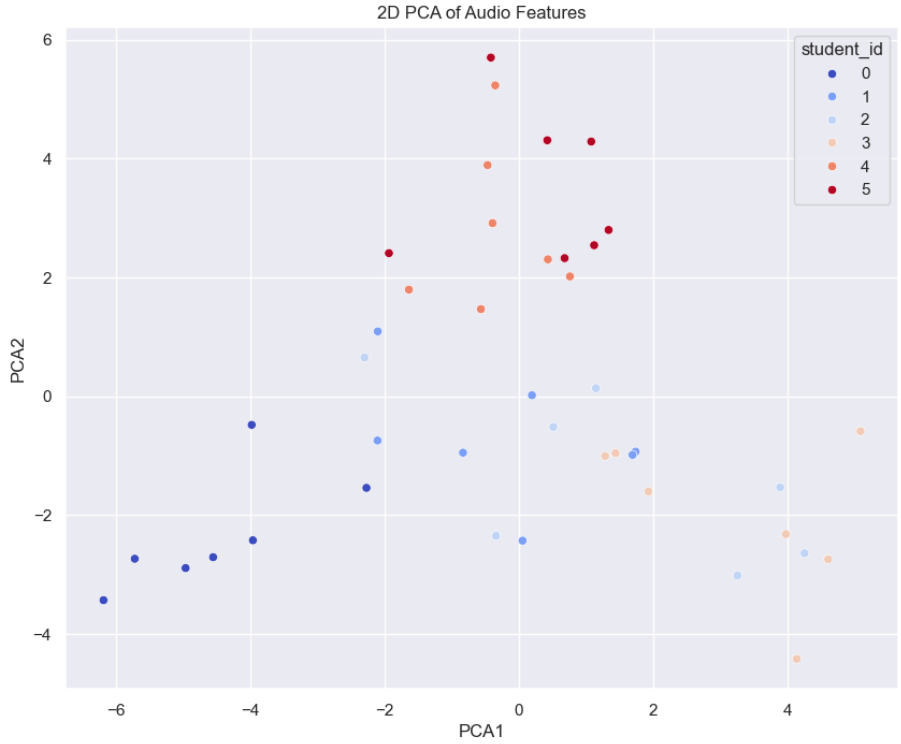


Figure 22: 2D PCA audio features

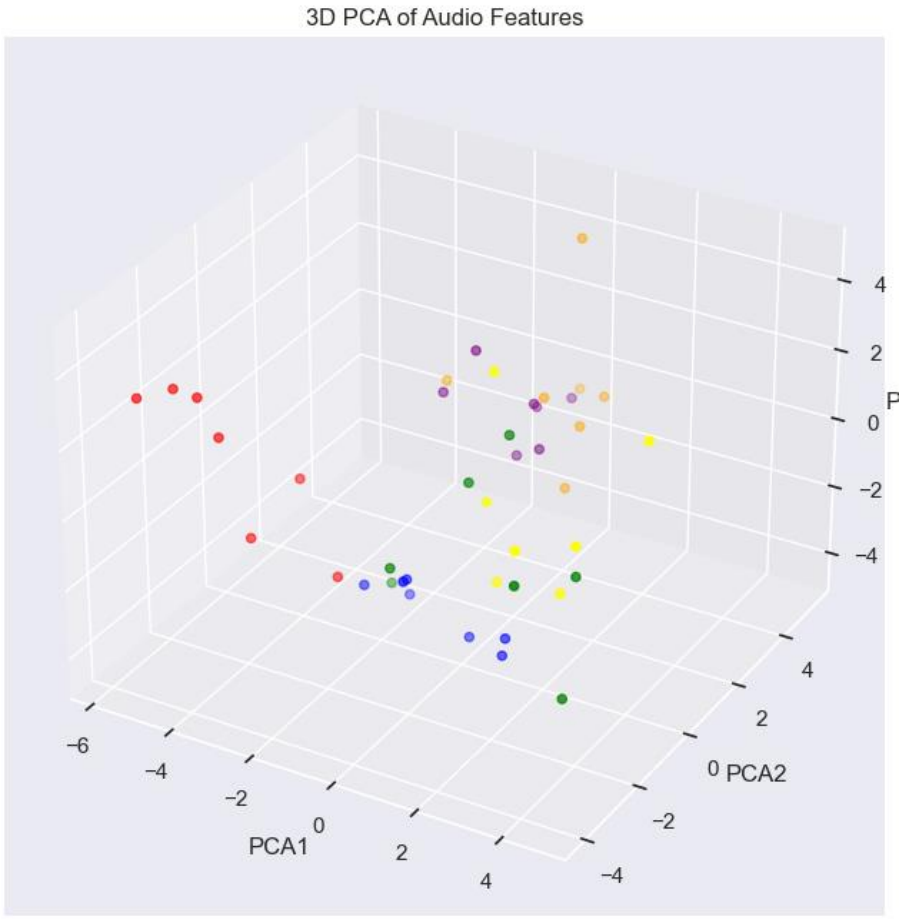


Figure 23: 3D PCA audio features

From the plots we can see that samples from same class are near each other.

Normalization

Again we normalize and split the data:

```
X = data.drop(columns=["student_id"])
y = data["student_id"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=12/42,
random_state=42, stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Now we have 30 samples for training and 12 for test.

Training Models

In these task, we used the following models:

- MLP works well if the dataset has non-linear patterns.

- SVM is great for small datasets with clear class boundaries.

- Random Forest can handle both linear & non-linear relationships.

- KNN is useful when data points of the same class are close together.

We train these models 3 times with 3 different set of data.

```
def train_and_evaluate_models(X_train, X_test, y_train, y_test):
    models = {
        "MLP": MLPClassifier(hidden_layer_sizes=(50, 30), max_iter=500, random_state=42),
        "SVM": SVC(kernel='linear', probability=True, random_state=42),
        "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
        "KNN": KNeighborsClassifier(n_neighbors=3)
    }
    results = {}
    for name, model in models.items():
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        acc = accuracy_score(y_test, y_pred)
        results[name] = acc
        print(f"{name} Accuracy: {acc:.4f}")
        print(classification_report(y_test, y_pred))
        cm = confusion_matrix(y_test, y_pred)
        plt.figure(figsize=(5, 4))
        sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
        plt.title(f"Confusion Matrix - {name}")
        plt.xlabel("Predicted")
        plt.ylabel("Actual")
```

```
        plt.show()
        all_classes = np.unique(y_train)  # Get all possible classes
        y_test_bin = label_binarize(y_test, classes=all_classes)  # Use full class set
        plt.figure(figsize=(8, 6))
        for i, class_label in enumerate(all_classes):
            if hasattr(model, "predict_proba"):
                y_prob = model.predict_proba(X_test)
            else:
                y_prob = model.decision_function(X_test)
                y_prob = (y_prob - y_prob.min()) / (y_prob.max() - y_prob.min())
            if np.sum(y_test_bin[:, i]) == 0:
                print(f"Skipping class {i} as there are no positive samples in the true
labels.")
                continue
            fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_prob[:, i])
            roc_auc = auc(fpr, tpr)
            plt.plot(fpr, tpr, lw=2, label=f"Class {i} (AUC = {roc_auc:.2f})")
        plt.plot([0, 1], [0, 1], color="gray", linestyle="--")
        plt.xlabel("False Positive Rate")
        plt.ylabel("True Positive Rate")
        plt.title("ROC Curve - Multi-Class")
        plt.legend(loc="lower right")
        plt.show()

    best_model = max(results, key=results.get)
    print(f"\n Best Model: {best_model} with Accuracy: {results[best_model]:.4f}")

    return results
```

The result of each model

Result of MLP model in first round:

```
MLP Accuracy: 1.0000
              precision    recall  f1-score   support
   810100091       1.00      1.00      1.00         2
   810100135       1.00      1.00      1.00         2
   810100150       1.00      1.00      1.00         2
   810100198       1.00      1.00      1.00         2
   810100270       1.00      1.00      1.00         2
   810102142       1.00      1.00      1.00         2

    accuracy                           1.00        12
   macro avg       1.00      1.00      1.00        12
weighted avg       1.00      1.00      1.00        12
```

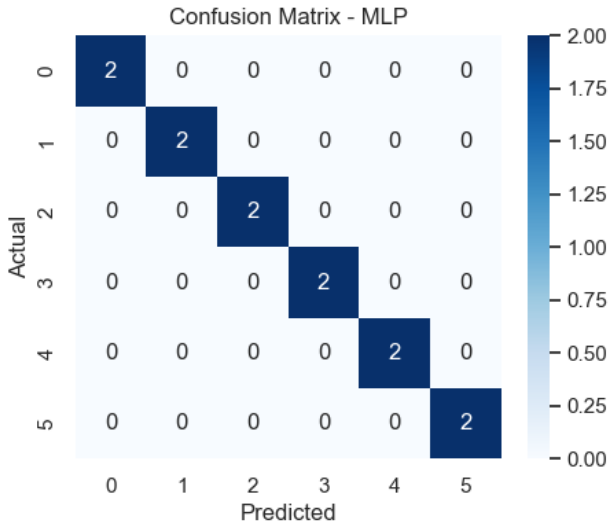


Figure 24: MLP confusion matrix in Identity Recognition

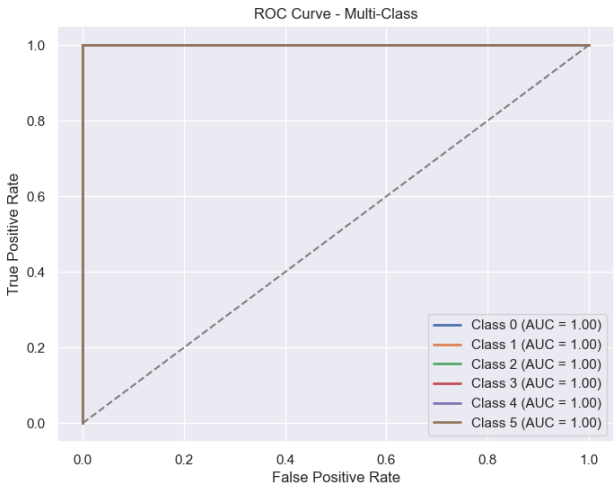


Figure 25: MLP ROC curve in Identity Recognition

Result of SVM model in first round:

```
SVM Accuracy: 0.9167
              precision    recall  f1-score   support

   810100091       1.00      1.00      1.00         2
   810100135       1.00      1.00      1.00         2
   810100150       1.00      1.00      1.00         2
   810100198       1.00      0.50      0.67         2
   810100270       0.67      1.00      0.80         2
   810102142       1.00      1.00      1.00         2

    accuracy                           0.92        12
   macro avg       0.94      0.92      0.91        12
weighted avg       0.94      0.92      0.91        12
```

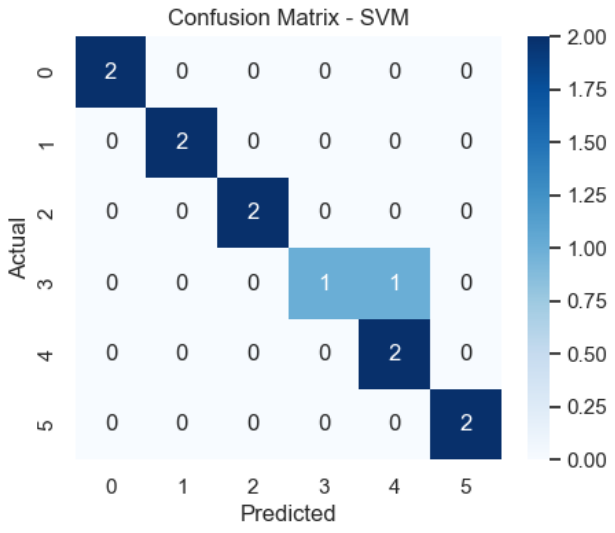


Figure 26: SVM confusion matrix in Identity Recognition

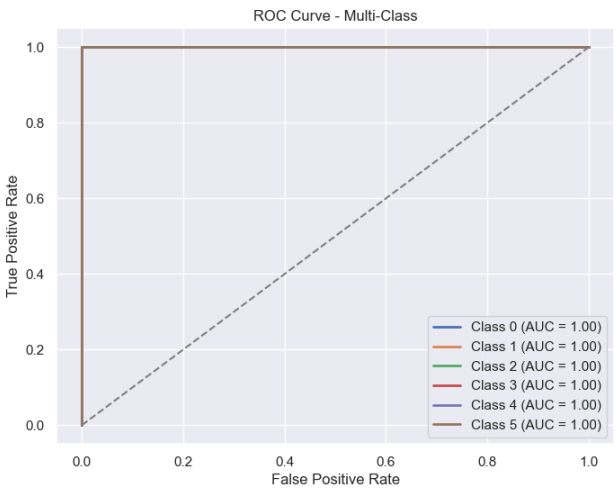


Figure 27: SVM ROC curve in Identity Recognition

Result of Random Forest model in first round:

```
Random Forest Accuracy: 0.9167
              precision    recall  f1-score   support
   810100091       0.67      1.00      0.80         2
   810100135       1.00      1.00      1.00         2
   810100150       1.00      1.00      1.00         2
   810100198       1.00      1.00      1.00         2
   810100270       1.00      1.00      1.00         2
   810102142       1.00      0.50      0.67         2

    accuracy                           0.92        12
   macro avg       0.94      0.92      0.91        12
weighted avg       0.94      0.92      0.91        12
```

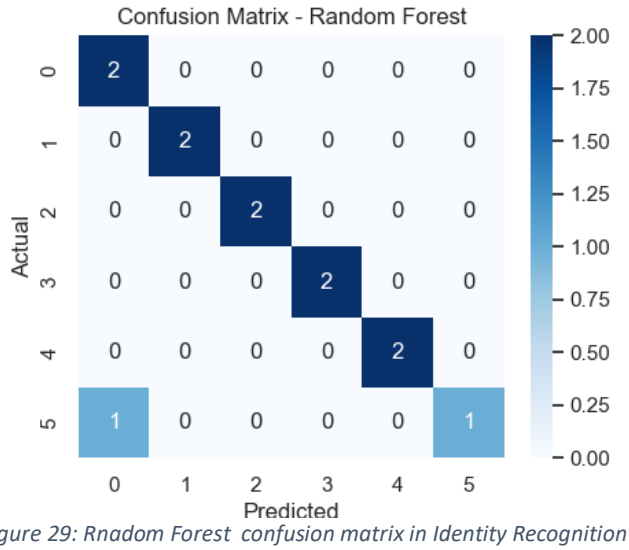


Figure 29: Rnadom Forest confusion matrix in Identity Recognition

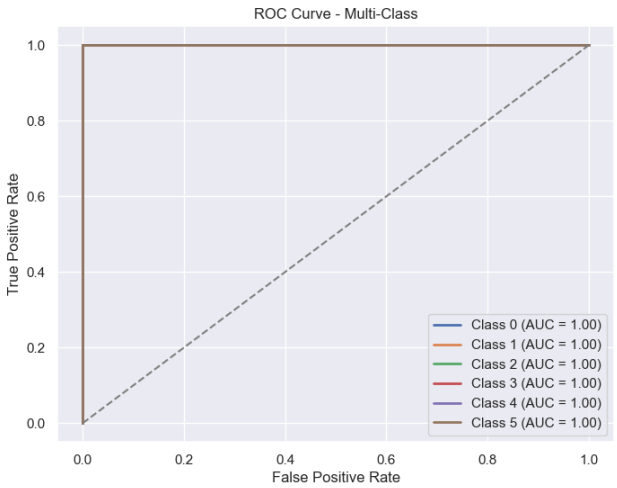


Figure 28: Random Forest ROC curve in Identity Recognition

Result of KNN model in first round:

```
KNN Accuracy: 0.8333
              precision    recall  f1-score   support
   810100091       1.00      1.00      1.00         2
   810100135       1.00      1.00      1.00         2
   810100150       0.67      1.00      0.80         2
   810100198       0.67      1.00      0.80         2
   810100270       1.00      0.50      0.67         2
   810102142       1.00      0.50      0.67         2

    accuracy                           0.83        12
   macro avg       0.89      0.83      0.82        12
weighted avg       0.89      0.83      0.82        12
```

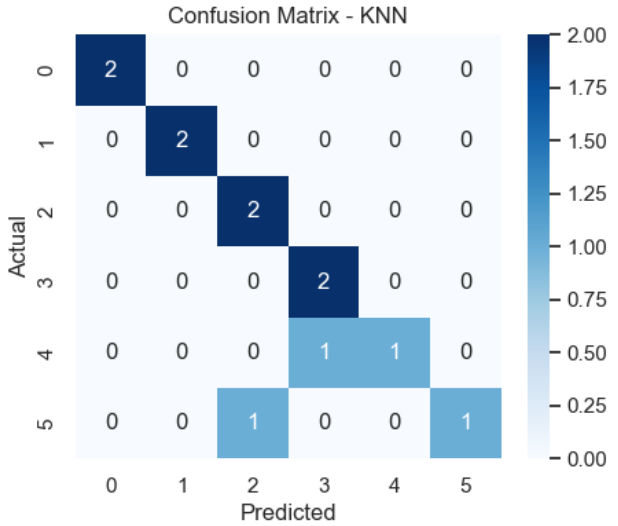


Figure 31: KNN confusion matrix in Identity Recognition

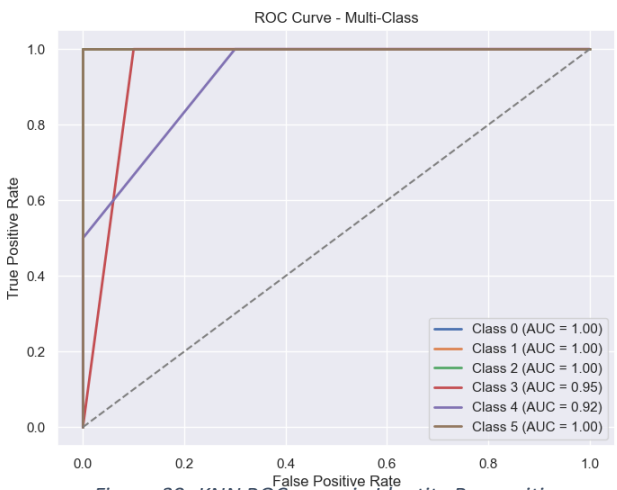


Figure 30: KNN ROC curve in Identity Recognition

In first round the best model is MLP:

```
 Best Model: MLP with Accuracy: 1.0000
{'MLP': 1.0,
 'SVM': 0.9166666666666666,
 'Random Forest': 0.9166666666666666,
 'KNN': 0.8333333333333334}
```

For each round we run this code to have different set of data and to train and evaluate the models:

```
data = get_6_students_data()
X = data.drop(columns=["student_id"])
y = data["student_id"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=12/42,
random_state=42, stratify=y)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
train_and_evaluate_models(X_train, X_test, y_train, y_test)
```

Result of MLP model in second round:

```
MLP Accuracy: 0.9167
              precision    recall  f1-score   support

   810100135       1.00      0.50      0.67         2
   810101423       1.00      1.00      1.00         2
   810101575       1.00      1.00      1.00         2
   810103162       0.67      1.00      0.80         2
   810103197       1.00      1.00      1.00         2
   810198500       1.00      1.00      1.00         2

    accuracy                           0.92        12
   macro avg       0.94      0.92      0.91        12
weighted avg       0.94      0.92      0.91        12
```

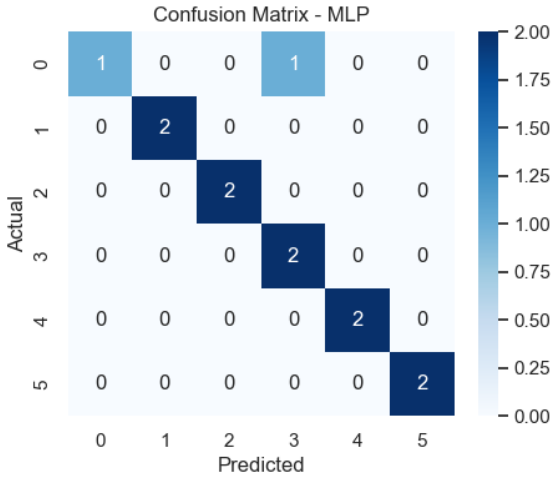


Figure 32: MLP confusion matrix in Identity Recognition

Result of SVM model in second round:

```
SVM Accuracy: 0.8333
              precision    recall  f1-score   support

   810100135       1.00      0.50      0.67         2
   810101423       0.50      1.00      0.67         2
   810101575       1.00      1.00      1.00         2
   810103162       1.00      0.50      0.67         2
   810103197       1.00      1.00      1.00         2
   810198500       1.00      1.00      1.00         2

    accuracy                           0.83        12
   macro avg       0.92      0.83      0.83        12
weighted avg       0.92      0.83      0.83        12
```

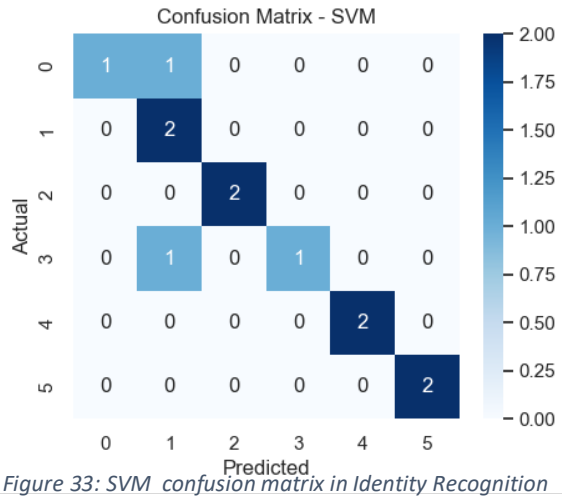


Figure 33: SVM confusion matrix in Identity Recognition

Result of Random Forest model in second round:

```
Random Forest Accuracy: 0.9167
              precision    recall  f1-score   support

   810100135       1.00      0.50      0.67         2
   810101423       1.00      1.00      1.00         2
   810101575       1.00      1.00      1.00         2
   810103162       1.00      1.00      1.00         2
   810103197       1.00      1.00      1.00         2
   810198500       0.67      1.00      0.80         2

    accuracy                           0.92        12
   macro avg       0.94      0.92      0.91        12
weighted avg       0.94      0.92      0.91        12
```

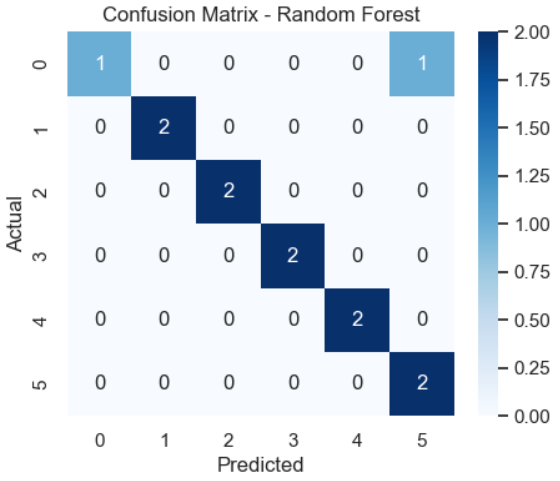


Figure 34: Rnadom Forest confusion matrix in Identity Recognition

Result of KNN model in second round:

```
KNN Accuracy: 0.9167
              precision    recall  f1-score   support

   810100135       1.00      1.00      1.00         2
   810101423       0.67      1.00      0.80         2
   810101575       1.00      1.00      1.00         2
   810103162       1.00      0.50      0.67         2
   810103197       1.00      1.00      1.00         2
   810198500       1.00      1.00      1.00         2

    accuracy                           0.92        12
   macro avg       0.94      0.92      0.91        12
weighted avg       0.94      0.92      0.91        12
```

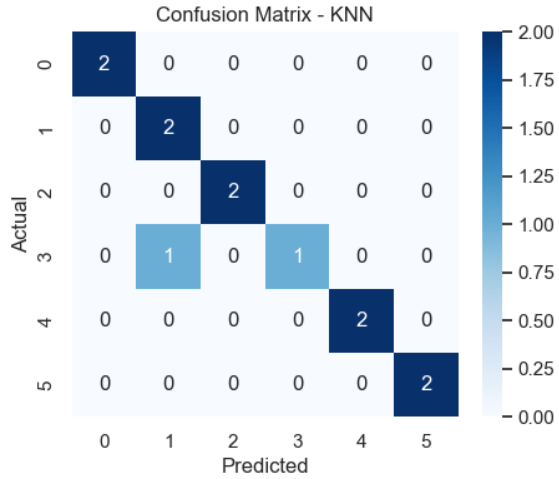


Figure 35: KNN confusion matrix in Identity Recognition

In second round the best model are MLP, Random Forest and KNN:

```
Best Model: MLP with Accuracy: 0.9167
{'MLP': 0.9166666666666666,
 'SVM': 0.8333333333333334,
 'Random Forest': 0.9166666666666666,
 'KNN': 0.9166666666666666}
```

Result of MLP model in third round:

```
MLP Accuracy: 0.9167
              precision    recall  f1-score   support

   810100268       0.67      1.00      0.80         2
   810101413       1.00      1.00      1.00         2
   810102017       1.00      1.00      1.00         2
   810103169       1.00      0.50      0.67         2
   810103322       1.00      1.00      1.00         2
   810199400       1.00      1.00      1.00         2

    accuracy                           0.92        12
   macro avg       0.94      0.92      0.91        12
weighted avg       0.94      0.92      0.91        12
```

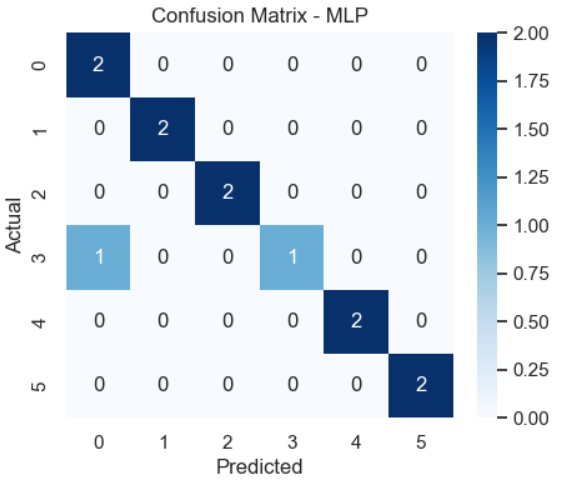


Figure 36: MLP confusion matrix in Identity Recognition

Result of SVM model in third round:

```
SVM Accuracy: 0.9167
              precision    recall  f1-score   support

   810100268       1.00      1.00      1.00         2
   810101413       1.00      1.00      1.00         2
   810102017       1.00      1.00      1.00         2
   810103169       1.00      0.50      0.67         2
   810103322       0.67      1.00      0.80         2
   810199400       1.00      1.00      1.00         2

    accuracy                           0.92        12
   macro avg       0.94      0.92      0.91        12
weighted avg       0.94      0.92      0.91        12
```

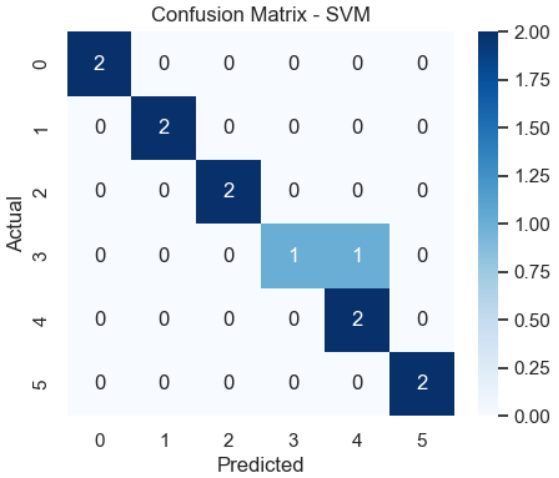


Figure 37: SVM confusion matrix in Identity Recognition

Result of Random Forest model in third round:

```
Random Forest Accuracy: 1.0000
              precision    recall  f1-score   support

   810100268       1.00      1.00      1.00         2
   810101413       1.00      1.00      1.00         2
   810102017       1.00      1.00      1.00         2
   810103169       1.00      1.00      1.00         2
   810103322       1.00      1.00      1.00         2
   810199400       1.00      1.00      1.00         2

    accuracy                           1.00        12
   macro avg       1.00      1.00      1.00        12
weighted avg       1.00      1.00      1.00        12
```

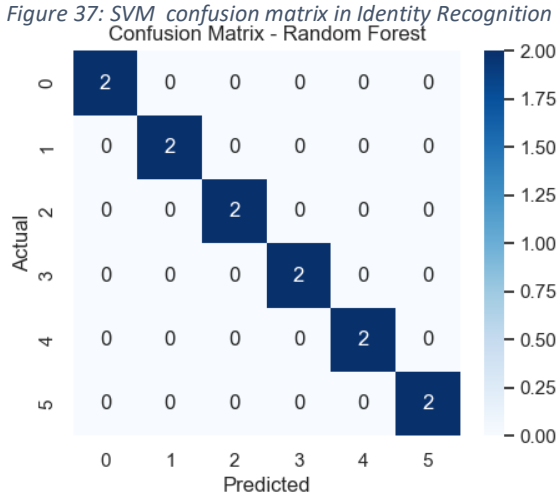


Figure 38: Random Forest confusion matrix in Identity Recognition

Result of KNN model in third round:

```
KNN Accuracy: 1.0000
              precision    recall  f1-score   support

   810100268       1.00      1.00      1.00         2
   810101413       1.00      1.00      1.00         2
   810102017       1.00      1.00      1.00         2
   810103169       1.00      1.00      1.00         2
   810103322       1.00      1.00      1.00         2
   810199400       1.00      1.00      1.00         2

    accuracy                           1.00        12
   macro avg       1.00      1.00      1.00        12
weighted avg       1.00      1.00      1.00        12
```

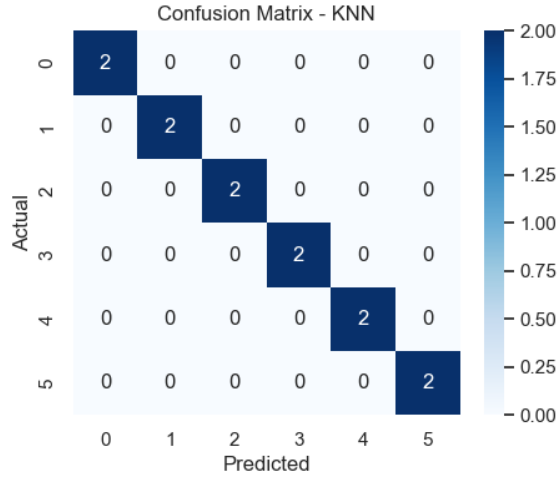


Figure 39: KNN confusion matrix in Identity Recognition

In third round the best model are Random Forest and KNN:

```
Best Model: Random Forest with Accuracy: 1.0000
{'MLP': 0.9166666666666666,
 'SVM': 0.9166666666666666,
 'Random Forest': 1.0,
 'KNN': 1.0}
```

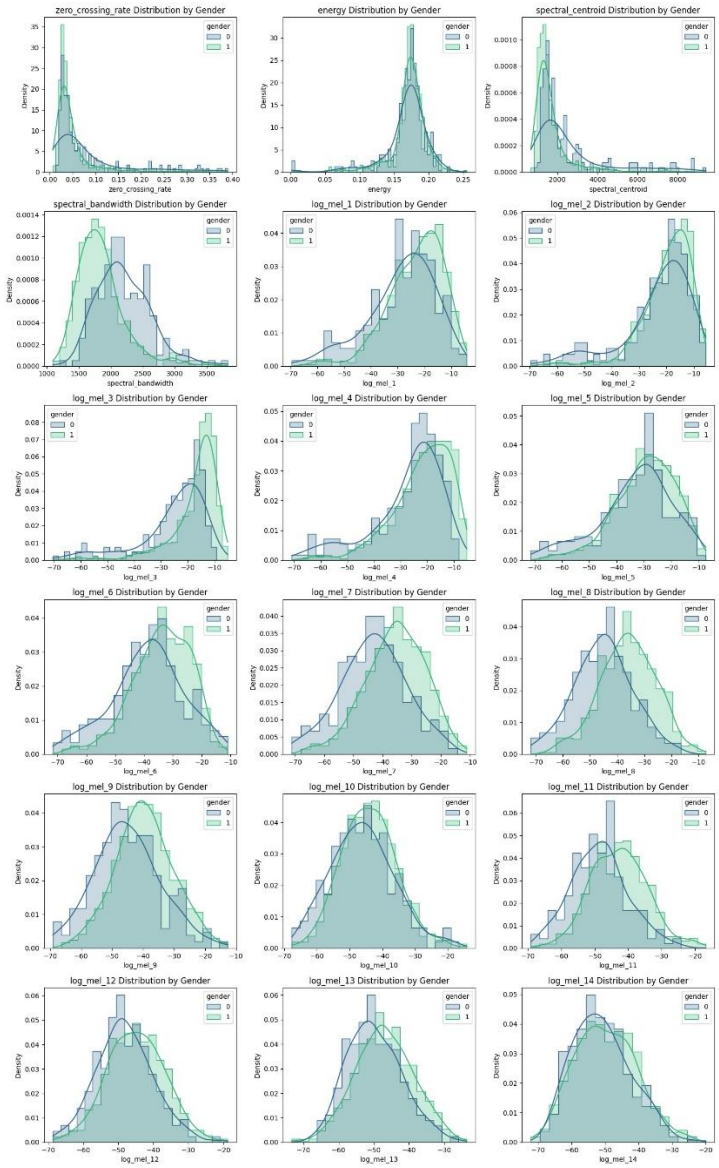
Clustering

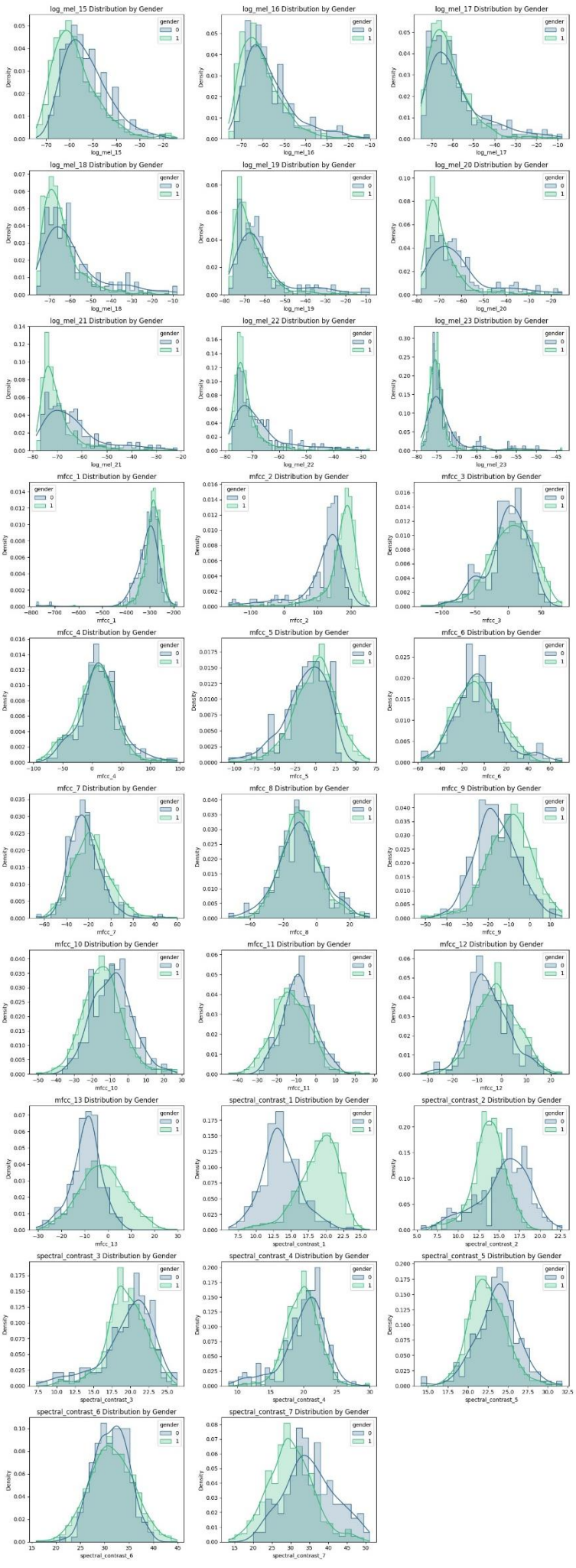
We aim to cluster our dataset. Our goal is to apply at least two clustering algorithms to the dataset. Determine the optimal number of clusters using the Silhouette Score. Analyze and compare the performance of the clustering models. We use K-Means Clustering, Agglomerative Clustering and, DBSCAN.

First, we load the data set and drop irrelevant features.

```
df = pd.read_csv('final_dataset.csv')
excluded_columns = ["question_number",
                    "student_id",
                    "has_intro",
                    "file_name",
                    "gender"]
data = df.copy()
df = df.drop(columns=excluded_columns)
```

In EDA step we have the following plots:




log_mel_15 Distribution by Gender
log_mel_16 Distribution by Gender
log_mel_17 Distribution by Gender
log_mel_18 Distribution by Gender
log_mel_19 Distribution by Gender
log_mel_20 Distribution by Gender
log_mel_21 Distribution by Gender
log_mel_22 Distribution by Gender
log_mel_23 Distribution by Gender
mfcc_1 Distribution by Gender
mfcc_2 Distribution by Gender
mfcc_3 Distribution by Gender
mfcc_4 Distribution by Gender
mfcc_5 Distribution by Gender
mfcc_6 Distribution by Gender
mfcc_7 Distribution by Gender
mfcc_8 Distribution by Gender
mfcc_9 Distribution by Gender
mfcc_10 Distribution by Gender
mfcc_11 Distribution by Gender
mfcc_12 Distribution by Gender
mfcc_13 Distribution by Gender
spectral_contrast_1 Distribution by Gender
spectral_contrast_2 Distribution by Gender
spectral_contrast_3 Distribution by Gender
spectral_contrast_4 Distribution by Gender
spectral_contrast_5 Distribution by Gender
spectral_contrast_6 Distribution by Gender
spectral_contrast_7 Distribution by Gender
gender
0
1
Density

Then we applied the steps in the last phase for feature selection. We compute correlations and applied PCA and dropped highly correlated features. The final number of features is 29. Then we normalized data and applied PCA for visualization.

```
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(scaled_data)
```

Before defining the models, we have defined a function to measure and report similarity and dissimilarity.

- Within-cluster similarity:

This refers to how similar the data points are to each other within the same cluster.

To measure similarity, you can use metrics such as Euclidean distance, cosine similarity, or Manhattan distance. For clustering methods like K-Means, you can calculate the average distance of each data point to the cluster centroid as a measure of similarity. Lower distance indicates higher similarity within the cluster.

Why data are in the same cluster: Data points are grouped together in the same cluster because they share similar characteristics. In K-Means clustering, for instance, data points are assigned to the cluster whose centroid is closest (in terms of the chosen distance metric). Clustering algorithms like DBSCAN or hierarchical clustering also group similar data points together based on density or proximity.

- Between-cluster dissimilarity:

This refers to how distinct or different the data points are from those in other clusters.

To measure dissimilarity between clusters, you can compute the distance between cluster centroids or use a more advanced metric like Silhouette Score, which considers both within-cluster cohesion and between-cluster separation.

Why some data are in a different cluster: Data points in different clusters are assigned to separate groups because they have sufficiently different characteristics, such as feature values or distributions. A large distance between the cluster centroids or a distinct boundary in feature space indicates that the data points from different clusters are dissimilar.

K-Means Clustering :

```
def euclidean_distance(a, b):
    return np.linalg.norm(a - b)


def similarity_and_dissimilarity_report(kmeans_model: KMeans):
  centroids = kmeans_model.cluster_centers_
  num_clusters = len(centroids)
  dissimilarity_matrix = np.zeros((num_clusters, num_clusters))

  for i in range(num_clusters):
    for j in range(num_clusters):
```

```
        if i != j:
            dissimilarity_matrix[i, j] = euclidean_distance(centroids[i], centroids[j])

    similarity_matrix = 1 / (1 + dissimilarity_matrix)

    print("Dissimilarity Matrix (Euclidean Distance):")
    print(dissimilarity_matrix)

    print("\nSimilarity Matrix (Inverse of Euclidean Distance):")
    print(similarity_matrix)

    print("\nMin and Max Dissimilarity:")
    print(f"Min: {np.min(dissimilarity_matrix)} | Max: {np.max(dissimilarity_matrix)}")

    print("\nMin and Max Similarity:")
    print(f"Min: {np.min(similarity_matrix)} | Max: {np.max(similarity_matrix)}")
```

This code calculates the similarity and dissimilarity between clusters in a K-Means clustering model based on the Euclidean distance between centroids. The function euclidean_distance(a, b) computes the Euclidean distance between two points. The main function, `similarity_and_dissimilarity_report(kmeans_model)`, first extracts the cluster centroids from the trained K-Means model and initializes a dissimilarity matrix, where each entry represents the Euclidean distance between two different cluster centroids. The similarity matrix is then computed as the inverse of the dissimilarity matrix (1 / (1 + distance)) to ensure similarity values are between 0 and 1. The function prints both matrices, along with the minimum and maximum values of dissimilarity and similarity, providing insights into how far apart or close the clusters are. This helps in evaluating the clustering quality—higher similarity values indicate clusters that are closer together, while higher dissimilarity values suggest well-separated clusters.

Then we use `silhouette_score` for different number of clusters in order to find the best number of cluster.

```
sil_scores = []
range_clusters = range(2, 10)

for n_clusters in range_clusters:
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    kmeans.fit(scaled_data)
    cluster_labels = kmeans.labels_

    score = silhouette_score(scaled_data, cluster_labels)
    sil_scores.append(score)

plt.plot(range_clusters, sil_scores, marker='o')
```

```
plt.title('Silhouette Scores for Different Numbers of Clusters')
plt.xlabel('Number of Clusters')
plt.ylabel('Silhouette Score')
plt.show()

best_n_clusters = range_clusters[np.argmax(sil_scores)]
print(f"The best number of clusters is: {best_n_clusters}")
```

This code determines the optimal number of clusters for K-Means clustering using the Silhouette Score, which measures how well-separated the clusters are. The loop iterates over different values of `n_clusters` (from 2 to 9), fits a K-Means model to the `scaled_data`, and calculates the silhouette score, which evaluates how close data points are to their own cluster compared to other clusters. The silhouette scores are stored in `sil_scores` and plotted against the number of clusters. The optimal cluster count is chosen as the one with the highest silhouette score, which is printed at the end. This helps in selecting a cluster number that maximizes intra-cluster cohesion while maintaining inter-cluster.

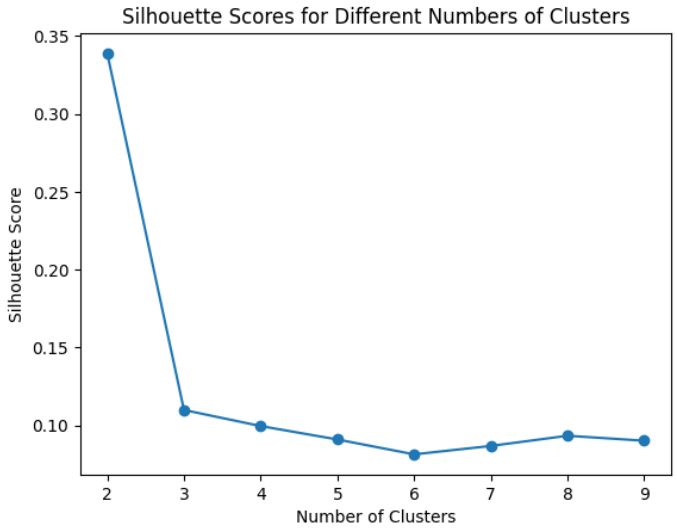


Figure 40

Based on the plot, the best number of clusters is 2.

Now we fit Kmeans model with K=2:

```
kmeans2 = KMeans(n_clusters=2, random_state=42)
kmeans2.fit(scaled_data)
plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=kmeans2.labels_, cmap='viridis',
alpha=0.7)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization of Clusters")
plt.legend(*scatter.legend_elements(), title="Cluster")
plt.show()
```

Now we want to analyze clusters identifying the feature that differs the most between clusters.

```
def analyze_clusters(data, kmeans_labels):
    data['cluster'] = kmeans_labels

    gender_counts = data.groupby(['cluster', 'gender']).size().unstack(fill_value=0)
    numeric_data = data.select_dtypes(include=[np.number])
    feature_means = numeric_data.groupby(data['cluster']).mean()

    print(gender_counts)
    print("energy",feature_means['energy'])

    feature_diffs = feature_means.diff().iloc[-1].abs()
    most_diff_feature = feature_diffs.idxmax()
    print(f"The feature with the most difference between clusters is: {most_diff_feature}
with difference:", feature_diffs.max())
    for cluster in range(len(np.unique(kmeans_labels))):
        print(f"Cluster {cluster} has {len(kmeans_labels[kmeans_labels == cluster])}
samples")
        print(f"{most_diff_feature} for cluster {cluster} is {feature_means.loc[cluster,
most_diff_feature]}")

analyze_clusters(data, kmeans2.labels_)
```

This function, `analyze_clusters()`, provides insights into the clusters formed by K-Means clustering by examining gender distribution, computing average feature values per cluster, and identifying the feature that differs the most between clusters.

Assign Cluster Labels: It adds a cluster column to the data data frame using `kmeans_labels`.

Gender Distribution per Cluster: It groups the data by cluster and gender, counts the occurrences, and prints a table showing how many male and female samples exist in each cluster.

Feature Averages per Cluster: It selects numeric features, groups them by cluster, and computes the mean value for each feature within each cluster.

Identifies the Most Differentiating Feature: The function calculates the absolute difference between the last two clusters' feature means (`diff().iloc[-1].abs()`) and identifies the feature with the highest difference.

Cluster Summary: It prints the size of each cluster and the average value of the most differentiating feature for each cluster.

This analysis helps understand how clusters differ, particularly which feature contributes most to separation, and whether gender plays a role in clustering.

- Result for K=2:

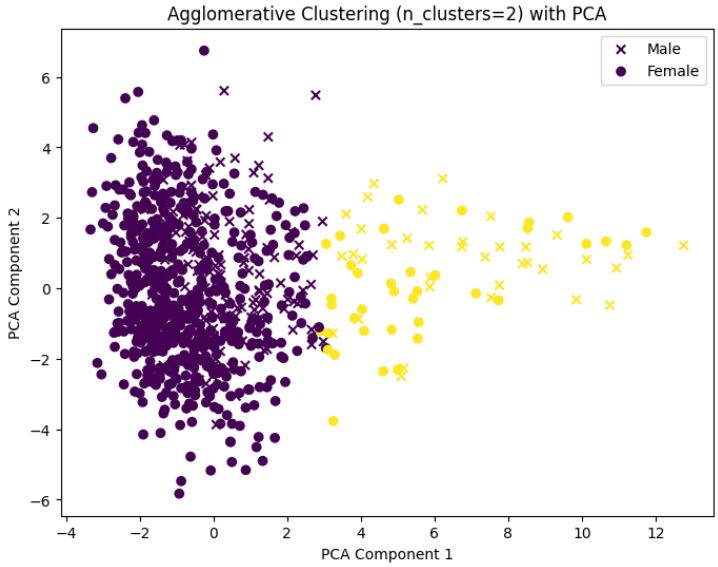


Figure 41

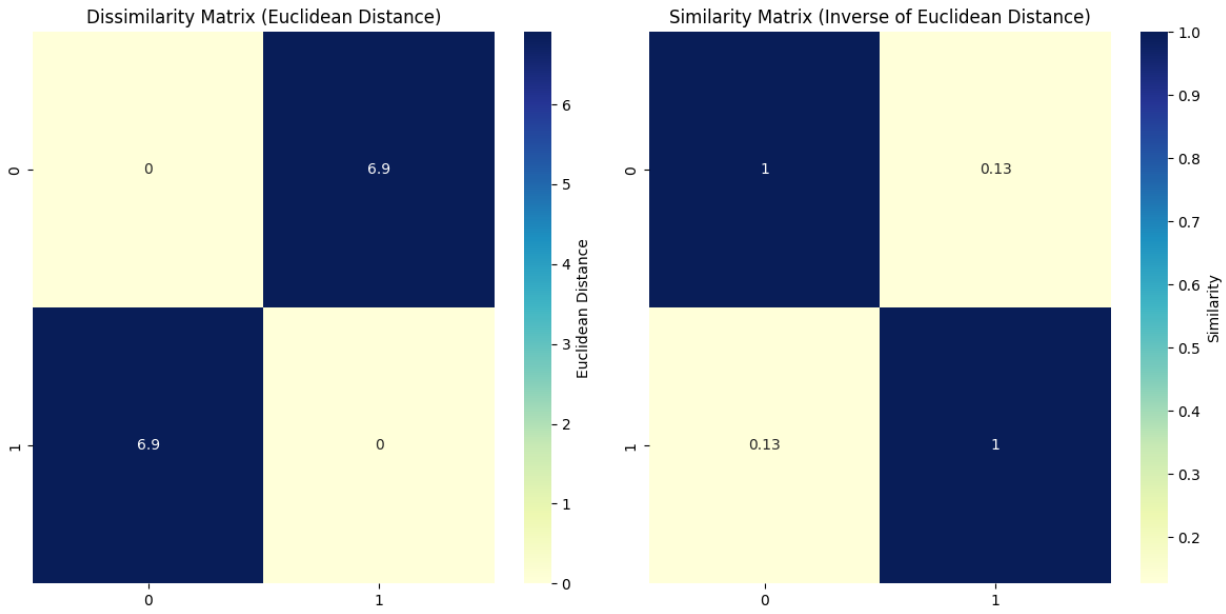


Figure 42

```
Min and Max Dissimilarity:
Min: 0.0 | Max: 6.905638640360468
Min and Max Similarity:
Min: 0.12649199457393914 | Max: 1.0
```

- Cluster 0 is male-dominated (593 males, 154 females), while Cluster 1 has a more balanced gender distribution (38 males, 36 females).

- Energy Levels: Cluster 0 has a higher average energy (0.173) than Cluster 1 (0.148), indicating that voices in Cluster 0 may be louder or more intense.

- Spectral Centroid Difference: The biggest distinguishing feature is spectral centroid, with Cluster 0 having lower values (1571.54 Hz) and Cluster 1 much higher (5138.66 Hz). This suggests Cluster 0 has lower-pitched voices, while Cluster 1 contains higher-pitched voices.

- Dissimilarity & Similarity: The Euclidean distance between clusters is 6.83, confirming that they are well-separated. The similarity score (0.127) indicates they are quite distinct.

- Conclusion: The clusters likely separate based on voice pitch, with Cluster 0 capturing lower-frequency, male-dominated voices, and Cluster 1 containing higher-frequency, more balanced voices.

- Result for K=4:

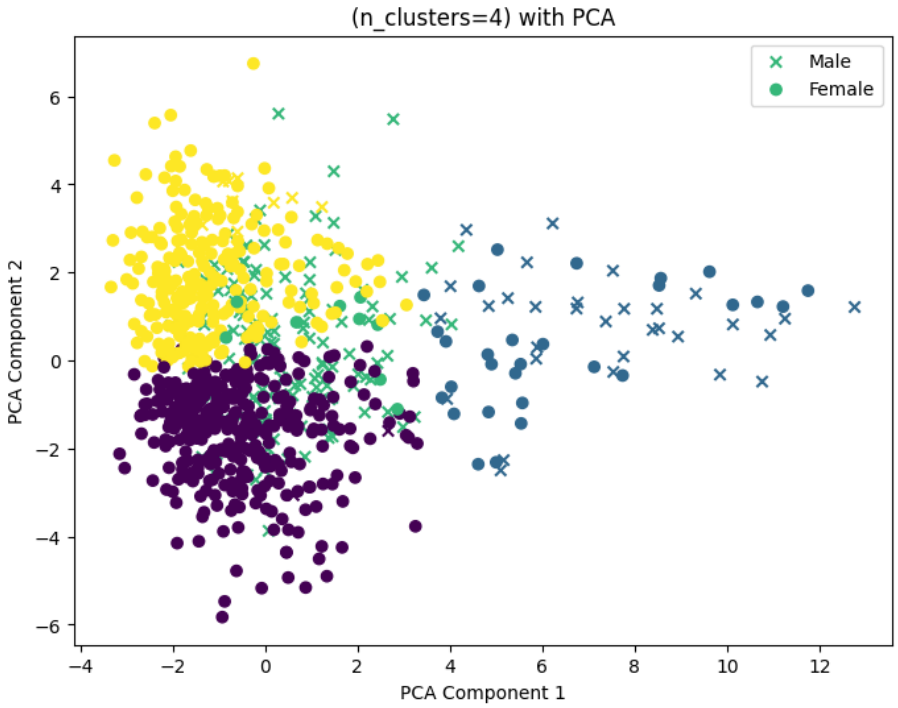


Figure 43

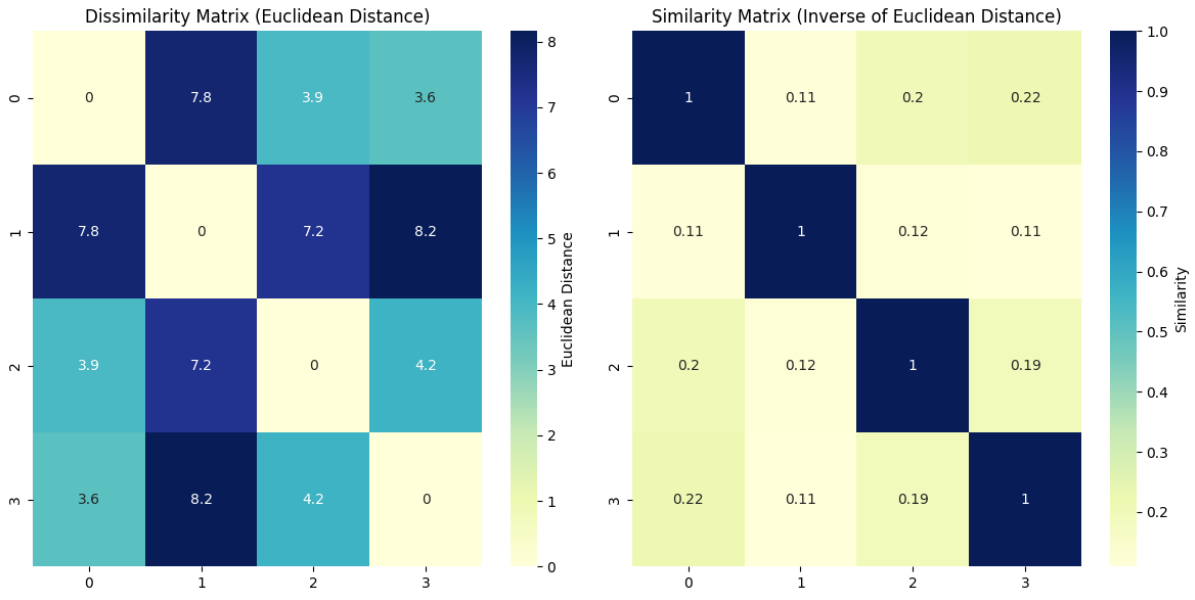


Figure 44

```
Min and Max Dissimilarity:
Min: 0.0 | Max: 8.159921708966866

Min and Max Similarity:
Min: 0.10917123876954932 | Max: 1.0
```

The clusters are primarily separated by pitch (spectral centroid) and energy levels. Cluster 1 has the highest pitch (6638 Hz) and lowest energy (0.158), making it the most distinct, likely capturing high-pitched voices. Cluster 0 has the lowest pitch (1361 Hz) and highest energy (0.176), suggesting deeper, stronger voices. Clusters 2 and 3 fall in between. Cluster 1 is the most dissimilar from others, as shown by the highest Euclidean distance (9.49). Most clusters are male-dominated, except Cluster 1, which is more balanced.

- Result for K=7:

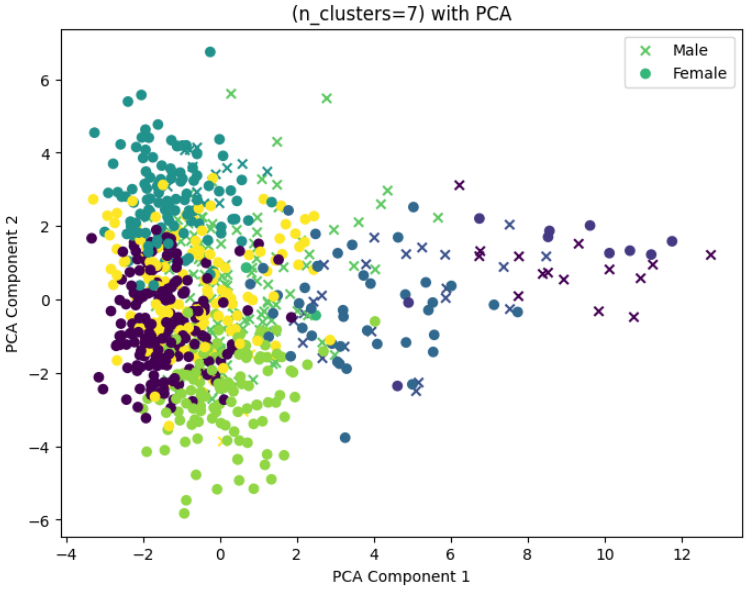


Figure 45

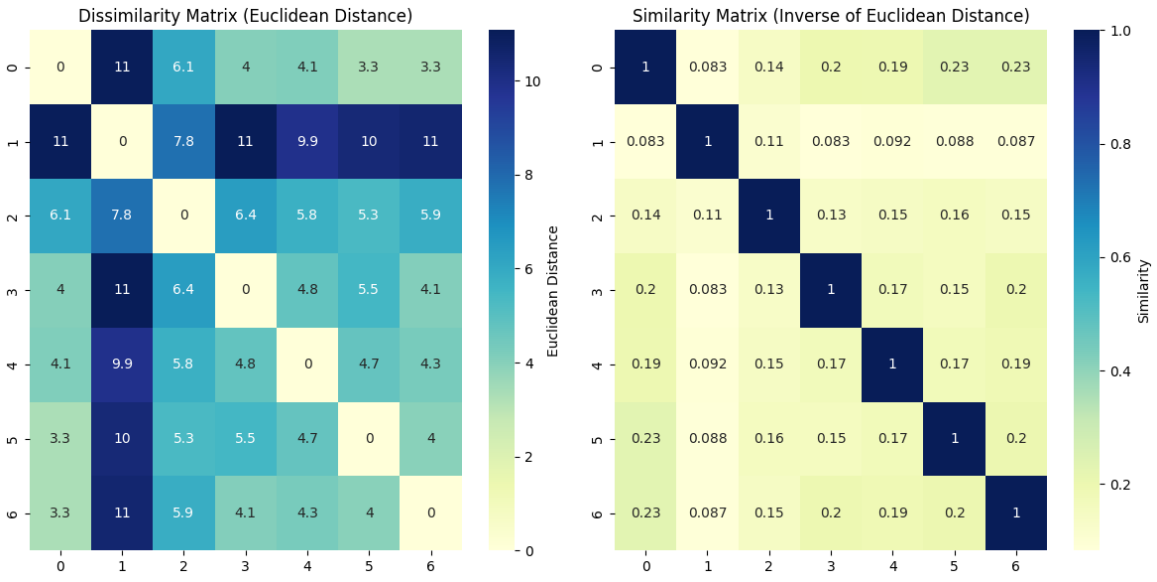


Figure 46

```
Min and Max Dissimilarity:
Min: 0.0 | Max: 11.077662221132504

Min and Max Similarity:
Min: 0.08279748031454977 | Max: 1.0
```

This 7-cluster KMeans analysis reveals strong differentiation based on spectral centroid (pitch) and energy levels. Cluster 1 is the most distinct, with the highest pitch (7559 Hz) and relatively low energy (0.164), likely representing high-pitched voices. Clusters 0, 2, 3, and 5 have similar low spectral centroid values (~1300-1500 Hz), suggesting deeper voices. Clusters 4 and 6 have moderate spectral centroids (2900 Hz and 3264 Hz), making them more distinct.

The dissimilarity matrix confirms Cluster 1 is the most different, with a maximum distance of 11.43 from Cluster 3, meaning it is highly separated from other groups. The highest similarity (0.25) is between Clusters 0 and 5, indicating they are quite similar in features.

In terms of gender distribution, most clusters have more male samples, except Cluster 4, which has more females (52 vs. 35), potentially indicating a gender-related feature influence.

Agglomerative Clustering

We changed the `similarity_and_dissimilarity_report` for this method, but the computations are the same.

```
def similarity_and_dissimilarity_report(agg_model: AgglomerativeClustering, data:
np.ndarray):
    if not hasattr(agg_model, 'labels_'):
        raise ValueError("AgglomerativeClustering model must be fitted before calling this
function.")
    labels = agg_model.labels_
    num_clusters = len(np.unique(labels))
    dissimilarity_matrix = np.zeros((num_clusters, num_clusters))
    for i in range(num_clusters):
        for j in range(num_clusters):
            if i != j:
                cluster_i = data[labels == i]
                cluster_j = data[labels == j]
                # Compute pairwise distance between two clusters (all pairs from cluster_i
and cluster_j)
                dissimilarity_matrix[i, j] = np.mean(pdist(np.vstack([cluster_i,
cluster_j])))
    # Similarity is the inverse of dissimilarity
    similarity_matrix = 1 / (1 + dissimilarity_matrix)
    print("Dissimilarity Matrix (Mean Pairwise Distance):")
    print(dissimilarity_matrix)

    print("\nSimilarity Matrix (Inverse of Mean Pairwise Distance):")
    print(similarity_matrix)

    print("\nMin and Max Dissimilarity:")
    print(f"Min: {np.min(dissimilarity_matrix[dissimilarity_matrix > 0])} | Max:
{np.max(dissimilarity_matrix)}")
```

```
    print("\nMin and Max Similarity:")
    print(f"Min: {np.min(similarity_matrix[similarity_matrix > 0])} | Max:
{np.max(similarity_matrix)}")
```

Again we plot `silhouette_score` for different number of clusters in order to find the best number of clusters.

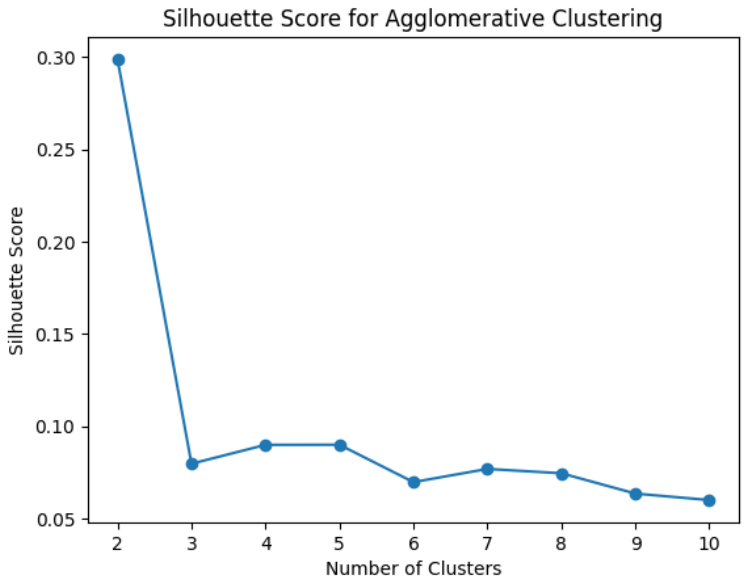


Figure 47

Based on the plot, the best number of clusters is 2.

- Result for n=2:

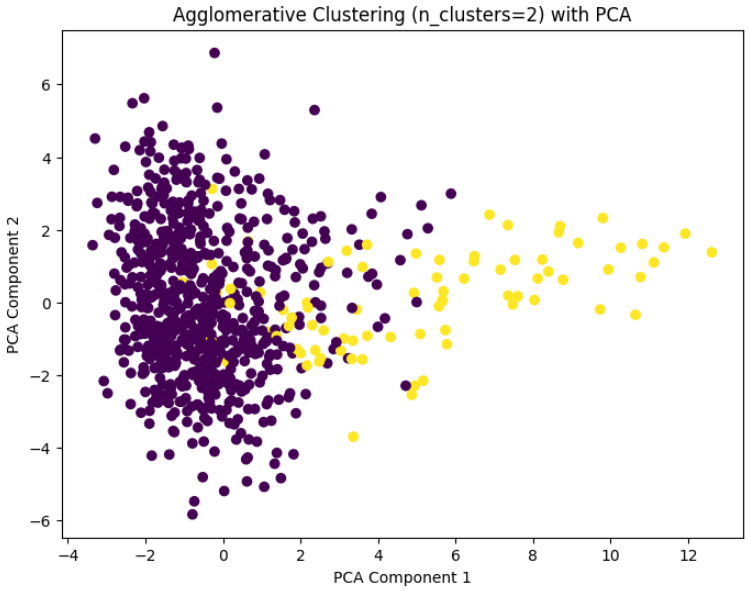


Figure 48

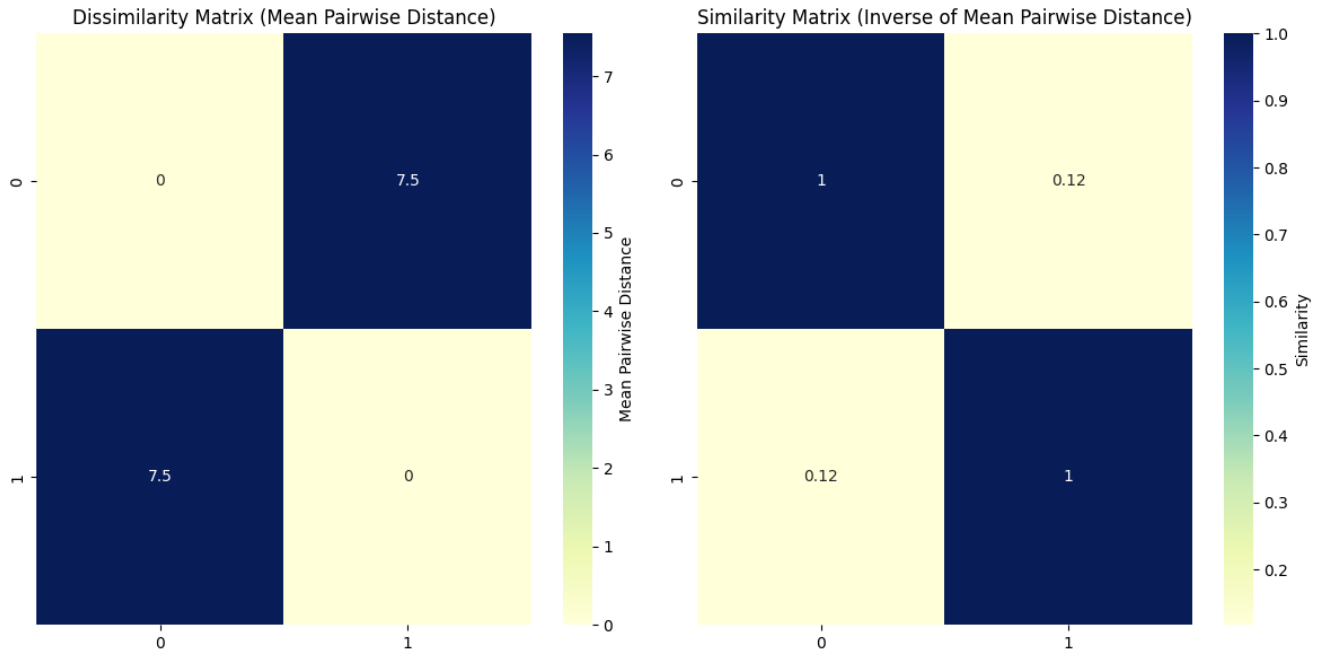


Figure 49

```
Min and Max Dissimilarity:

Min: 7.54340671653724 | Max: 7.54340671653724

Min and Max Similarity:
Min: 0.11704932624409968 | Max: 1.0
```

In Agglomerative Clustering (n=2):

- Cluster 0 has 799 samples (mostly male) and represents low-pitched voices (spectral centroid around 1737 Hz).

- Cluster 1 has 22 samples (mostly female) and represents high-pitched voices (spectral centroid around 7559 Hz).

- The dissimilarity between clusters is 7.54, indicating clear separation.

- The similarity is low (0.117), confirming the clusters are distinct.

- The most distinguishing feature is the spectral centroid, with a significant difference of 5822 Hz.

- Result for n=4:

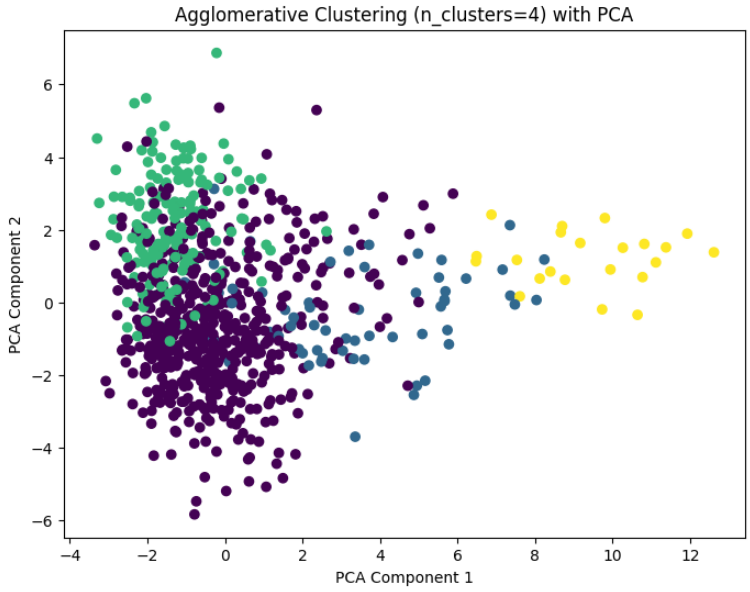


Figure 50

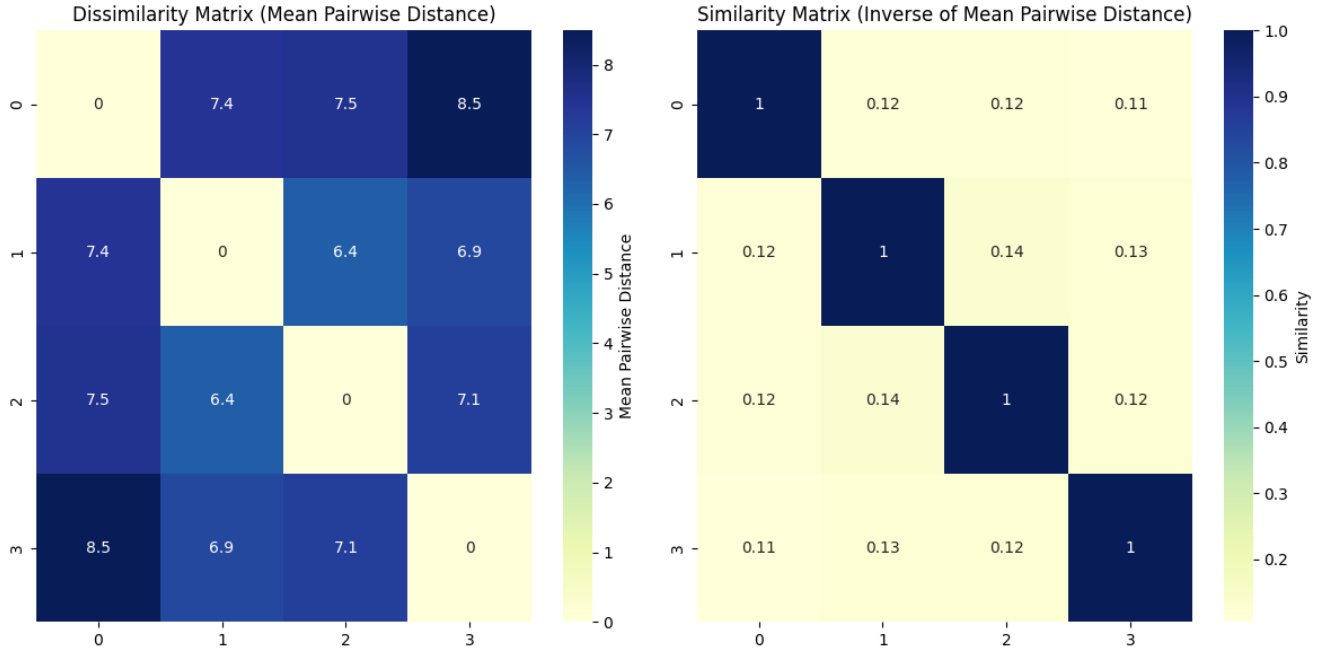


Figure 51

```
Min and Max Dissimilarity:
Min: 6.376515669195792 | Max: 8.489794911360578

Min and Max Similarity:
Min: 0.10537635526800096 | Max: 1.0
```

In this Agglomerative Clustering (n=4) result:

- Cluster 0 has 297 samples, mostly male, with a spectral centroid around 2312 Hz, indicating mid-range pitch voices.

- Cluster 1 has 255 samples, mostly female, with a spectral centroid around 1370 Hz, indicating lower-pitched voices.

- Cluster 2 has 247 samples, mostly female, with a spectral centroid around 1424 Hz, indicating slightly higher-pitched voices.

- Cluster 3 has 22 samples, mostly female, with a very high spectral centroid around 7559 Hz, indicating very high-pitched voices.

- Dissimilarity between clusters ranges from 6.38 to 8.49, showing clear separation between the clusters.

- Similarity is low (around 0.105 to 0.135), reinforcing that these clusters are distinct.

- The most significant feature differentiating clusters is spectral centroid, with a large difference of 6136 Hz, particularly between cluster 0 and cluster 3.

- Result for n=7:

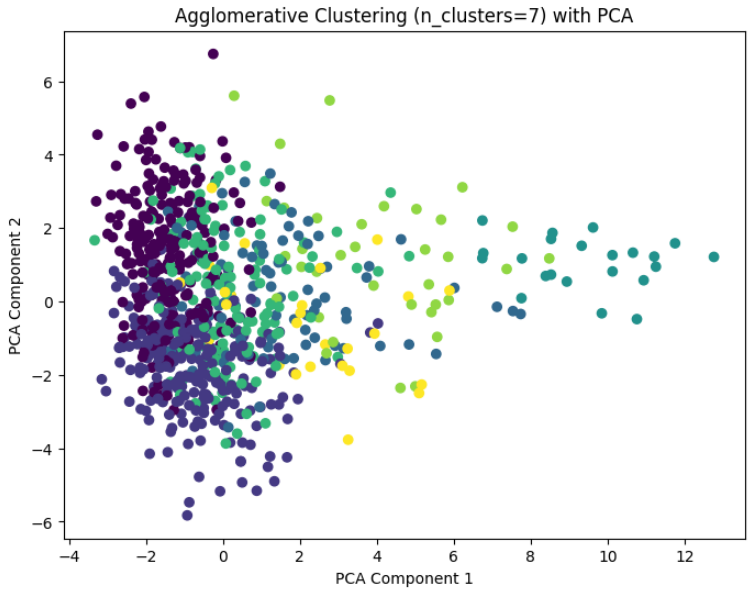


Figure 52

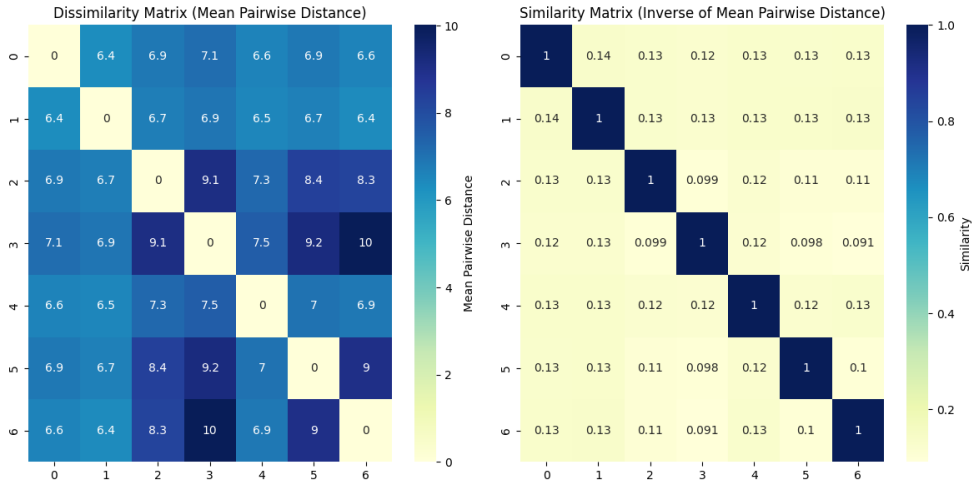


Figure 53

```
Min and Max Dissimilarity:
Min: 6.376515669195792 | Max: 10.017054270800887

Min and Max Similarity:
```

```
Min: 0.090768364702564443 | Max: 1.0
```

In this Agglomerative Clustering (n=7) result:

Cluster 0 (247 samples) and Cluster 1 (255 samples) are quite similar, both with spectral centroids around 1370-1423 Hz, representing mid-range pitch voices.

Cluster 2 (85 samples) has a significantly higher spectral centroid at 2530 Hz, indicating higher-pitched voices.

Cluster 3 (22 samples) has a very high spectral centroid at 7559 Hz, indicating very high-pitched voices.

Cluster 4 (149 samples) is more centered around 1795 Hz, representing voices that are lower than cluster 2 but higher than clusters 0 and 1.

Cluster 5 (35 samples) has a spectral centroid around 4199 Hz, indicating voices with a higher pitch compared to cluster 0 and 1.

Cluster 6 (28 samples) has a spectral centroid of 2042 Hz, indicating a moderate-high pitch.

Dissimilarity between clusters ranges from 6.38 to 10.02, indicating moderate to high separation between the clusters.

Similarity ranges from 0.090 to 1.0, with clusters that are farther apart having low similarity values.

The most significant feature differentiating the clusters is spectral centroid, with a large difference of 2156 Hz, particularly between cluster 0 (mid-range voices) and cluster 3 (very high-pitched voices).

DBSCAN

```
dbscan = DBSCAN(eps=0.5, min_samples=5)

dbscan_labels = dbscan.fit_predict(scaled_data)

plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=dbscan_labels, cmap='viridis', marker='o')
plt.title('DBSCAN Clustering with PCA')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.show()
```

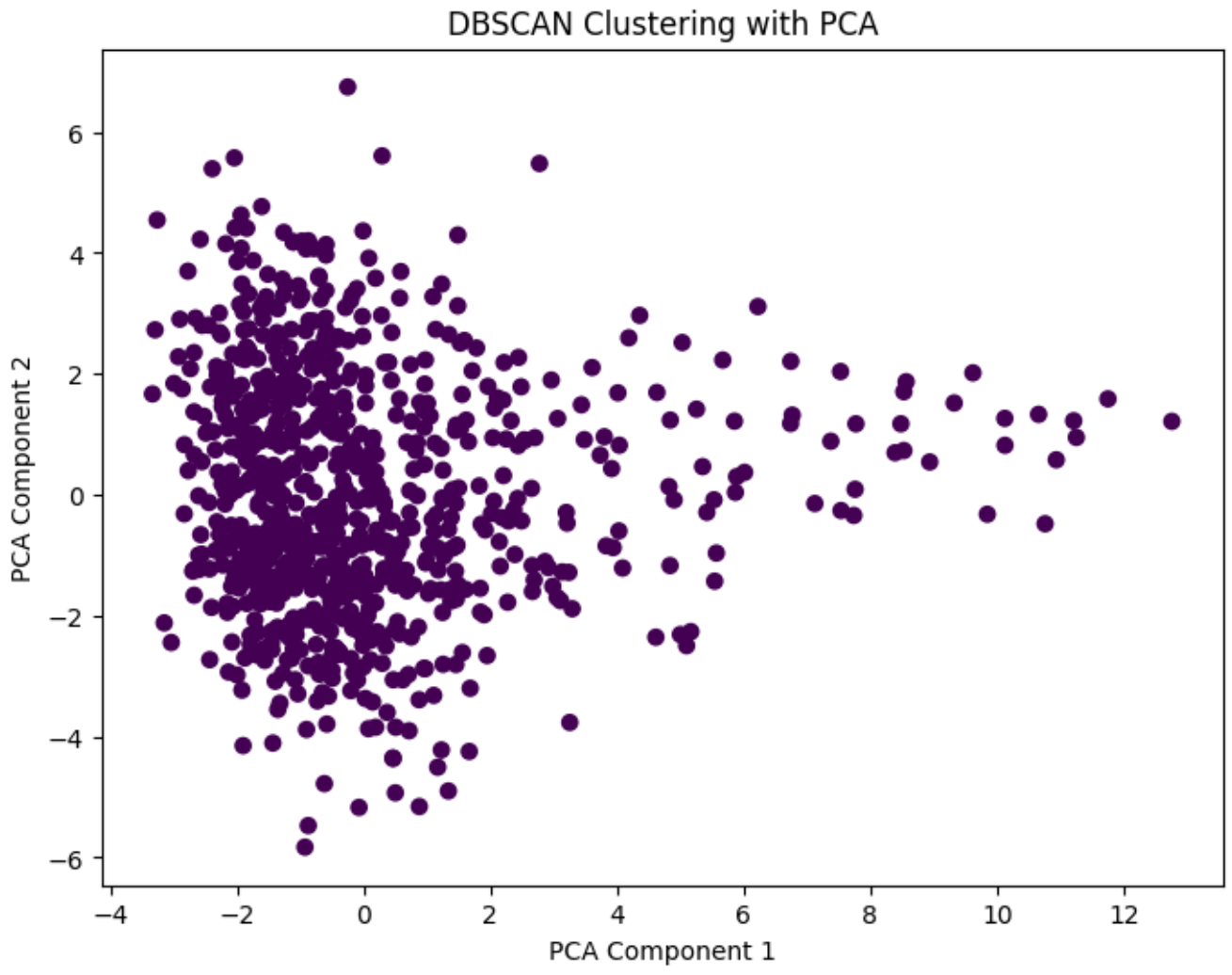


Figure 54